

Flushes Suffice: A Theory of Stateless PIFO Trees

Codex

Claude*

July 2026

Abstract

A packet scheduler is often easiest to observe in a batch experiment: enqueue a word of packets and then drain the scheduler. A stronger experiment may interleave enqueues and dequeues. For finite, stateless schedulers assembled from push-in first-out (PIFO) trees, we prove that the two observations have exactly the same discriminating power. If two valid schedulers agree on every flush experiment, then they agree on every interleaved operation word. The converse is immediate.

The proof exposes the extensional structure of a stateless PIFO tree. Binary flushes recover pair comparisons; a colored-PIFO coupling, a three-cell diamond, and fixed-head chords produce a meet-compatible preorder; swapping before refinement and collapsing equal selector levels completes the alphabet-inductive proof. A small stateful construction agrees on every flush but is separated by a five-operation interleaving, showing that fixed assignment is load-bearing. A canonical obstruction-component tree is a complete invariant. On a unit-cost RAM, it yields an $O(k^3h)$ structural decision procedure and a $\Theta(k^3)$ two-or-three-push test family whose triple-support count is optimal among suites using at most three types per test. A pair-dependent eta-style comparator instead avoids normal forms and returns either a two- or three-push witness or a finite proof DAG. The natural bidirectional rewrite inventory is equationally locally confluent but nonterminating—even a canonical tree is reducible—whereas a summary-carrying bottom-up fold computes the canonical form. Meet promotion removes induction on alphabet size and, by finite support, extends the observational theorem to arbitrary alphabets. Normalization by evaluation reifies the literal flush transformer and gives another finite-alphabet proof, though not an algorithmic normalizer. A pinned clean-slate manifest records kernel-checked Lean artifacts for the main result over a frozen statement, with per-artifact independence and axiom scopes.

1 Introduction

A scheduler can be questioned in two ways. A *flush* experiment first pushes a finite word of packets and then pops until every pushed packet has been returned. An *interleaved* experiment may alternate pushes and pops arbitrarily. Interleaving is apparently more informative: an early pop can change which packets remain together, and later arrivals then meet a residual state that no batch drain exposes directly. This raises a basic extensional question for packet schedulers:

If two schedulers produce the same drain for every input word, must they produce the same observations for every interleaving of pushes and pops?

For arbitrary stateful assignment policies, the answer is no. For stateless PIFO trees, the answer is yes. Here a PIFO is a priority queue ordered by rank, with FIFO order among equal ranks. A PIFO tree places such a queue at every node: an internal queue chooses a child, and a leaf queue chooses a packet. A push follows a root-to-leaf path and inserts one ranked entry at every visited node. *Stateless* means that the packet type alone fixes this entire annotated path. It may not depend on the current queues, the arrival number, or earlier operations.

*Authored by Codex and Claude.

Our main theorem says that, over every finite packet alphabet and arbitrary finite tree topologies, flush equivalence implies interleaved equivalence for valid stateless assignments. Since a flush is itself an operation word, the reverse implication is immediate. Thus flushes and general interleavings coincide extensionally on this class.

This result gives precise content to the practice of testing a scheduler by batch drains. As stated, its premise quantifies over all finite input words: the implication first says that no additional observational principle is gained by admitting residual-state experiments once all batch behavior is known. The later canonical analysis strengthens this with a finite test theorem. This distinction matters both conceptually and formally. The main proof reduces an operational equivalence over mixed commands to equality of a pure word transformer, without assuming that either scheduler’s hidden topology or root partition can be reconstructed from observations.

The proof has four ingredients.

1. On two packet types, a stateless PIFO tree is a single stable comparison. Two orderings of the binary flush distinguish strict priority in either direction from a FIFO tie.
2. A colored-PIFO coupling shows that changing ranks within colors preserves the complete online color trace whenever all cross-color comparisons are unchanged.
3. The top partitions of two flush-equivalent trees induce an incidence grid. Binary comparisons agree wherever both partitions separate a pair. A three-cell diamond rules out local comparison cycles, and a fixed-head-chord induction rules out cycles of every length. The result is one total preorder compatible with both roots across their blocks.
4. Both roots are first swapped to this common preorder. Each proper block is then refined into meet cells and discharged by strong induction on the alphabet. The remaining two selector levels use identical keys and collapse occurrence-for-occurrence.

The same local structure yields more than the implication. Effective comparisons on pairs and rooted obstruction bits on triples generate a canonical component-decomposition tree, which is a complete invariant for interleaved behavior. On a unit-cost RAM, they give an $O(k^3h)$ structural decision procedure, an injective test suite using at most three pushes, and a matching $\Theta(k^3)$ lower bound on triple supports among suites whose tests use at most three types. An independent eta-style comparator decides equivalence directly through pair-dependent meet obligations, without constructing either normal form; it returns a two- or three-push counterexample or a finite equivalence proof DAG. The natural active rewrite equations do not provide a rewrite normal form: collapse/regrouping and leaf explosion create cycles, and even a canonical tree is reducible. Nevertheless a compositional bottom-up fold over restriction-ready pair/triple summaries computes exactly the canonical form. A normalization-by-evaluation argument instead evaluates to the literal flush transformer, reifies its finest factor hierarchy, and gives a second semantic proof, not a decision algorithm. Finally, a separate meet-promotion argument removes induction on the number of packet types; by finite support, the observational theorem applies to arbitrary type alphabets over finite topologies, without asserting a global infinite-alphabet normal form.

Statelessness is sharp as a hypothesis on the class. We exhibit two flat, arrival-dependent assignment policies whose flush transformer is identical on every word but whose outputs differ after `push; pop; push; push; pop`. The example isolates the failure mode: an internal reference selects a leaf, not the packet occurrence that created that reference. A prior pop can remove the packet that would otherwise have been “hijacked” by a later reference. A fixed type-to-path assignment rules out exactly the overtaking freedom used by this construction.

Besides proving the equivalence theorem, the argument gives a useful semantic picture of stateless PIFO trees. They are arbitration hierarchies. Each pair of types meets at a first decisive queue, while the higher queues schedule only coarser child colors. Unary forwarding depth is operationally invisible, and consecutive selectors with the same preorder can be collapsed. This picture is developed in Section 5 after the formal proof.

The mathematical model in Section 2 is a direct presentation of the frozen Lean statement. Section 3 gives the complete proof, including the binary recovery table, colored coupling, meet construction, and collapse.

Section 4 works through the stateful separation. The later mechanization section records statement hashes, clean-slate proof checking, per-artifact independence, axiom audits, and the exact scope of the kernel witnesses. The subsequent sections develop canonical normalization, structural and eta-style decision, the negative rewrite result and positive bottom-up normalizer, normalization by evaluation, and topology-recursive proofs with their finite-support extension.

2 The model

This section presents the mathematical content of the frozen statement file. In particular, ranks and arrival stamps are natural numbers, topologies are finite ordered trees, a push carries one global stamp through the entire path, and a failed pop is observable and atomic.

2.1 Entries and PIFOs

Let X be a set of values. A queue entry is a triple

$$e = (\text{val}(e), \text{rank}(e), \text{arr}(e)) \in X \times \mathbb{N} \times \mathbb{N}.$$

The value is either a packet type or a child index, depending on the node at which the entry resides. Lower ranks have higher priority. The strict selection relation is lexicographic:

$$e \prec e' \iff \text{rank}(e) < \text{rank}(e') \vee (\text{rank}(e) = \text{rank}(e') \wedge \text{arr}(e) < \text{arr}(e')). \quad (1)$$

A PIFO queue is represented as a list in insertion order. Popping removes the least entry under Equation 1 and preserves the relative list order of the rest. Every push receives a fresh global arrival stamp and contributes at most one entry to a particular node. Consequently arrival stamps within a reachable queue are distinct, so its minimum is unique. The arrival component implements FIFO tie breaking among entries of equal rank.

2.2 Topologies, states, and paths

A topology is generated by

$$\tau ::= \text{leaf} \mid \text{node}[\tau_0, \dots, \tau_{m-1}],$$

where the child list is finite, ordered, and possibly empty. A state over packet values A has a queue at every node. A leaf queue contains entries whose values lie in A ; an internal queue contains entries whose values are natural-number child indices. The empty state has the given topology and an empty queue at every node.

An annotated root-to-leaf path is

$$p ::= \text{leaf}(r) \mid \text{node}(c, r, p), \quad c, r \in \mathbb{N}.$$

It is well formed for a topology when the constructors match at every level, each c is an in-range child index, and the remaining path fits that child. Pushing packet a with global arrival stamp j along a valid path appends one entry at each visited node:

$$\begin{array}{ll} \text{node}(c, r, p) : & (c, r, j) \quad \text{is appended to the internal queue, then the push follows child } c, \\ \text{leaf}(r) : & (a, r, j) \quad \text{is appended to the leaf queue.} \end{array}$$

Thus every entry created by one source push has the same arrival stamp. The formal definition also specifies ill-formed pushes, but the main theorem quantifies only over valid schedulers, so those branches are unreachable.

To pop a tree, pop its root queue. At a leaf, the selected value is emitted. At an internal node, the selected value is interpreted as a child index and the pop continues recursively in that child. The operation is *atomic*: if the root queue is empty, an index is invalid, or any recursive pop fails, the tree-level result is **none**, with no updated state exposed to the caller. For valid reachable states, the latter two cases do not arise; the balance argument appears in Lemma 3.1.

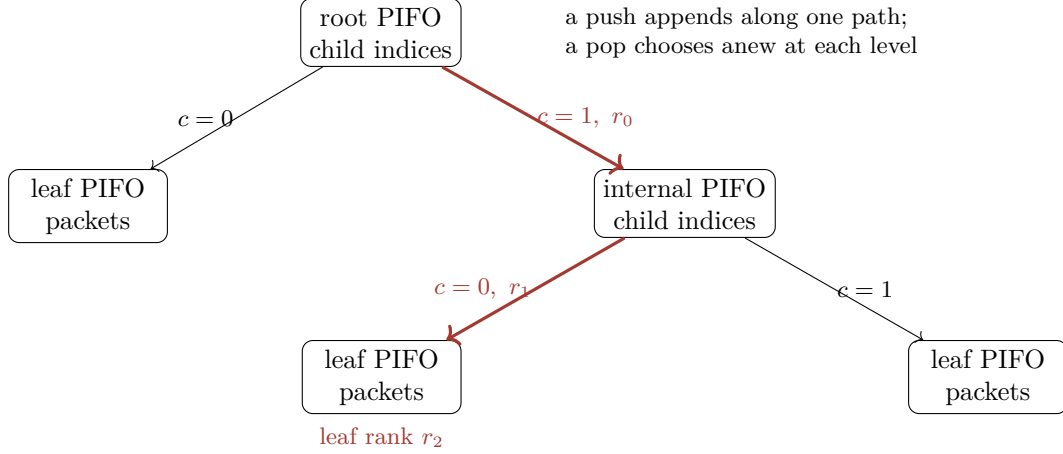


Figure 1: A PIFO-tree state. Internal entries name children; leaf entries name packets. The highlighted annotated path installs ranks r_0, r_1, r_2 with one shared arrival stamp. A later pop follows whichever child entry is then minimal; it need not follow a single source push's path.

2.3 Stateless schedulers and observations

Fix $k \in \mathbb{N}$ and the finite packet alphabet $A = \text{Fin}(k)$. A scheduler is a topology together with an assignment

$$S = (\tau, \text{assign}), \quad \text{assign} : A \rightarrow \text{Path}.$$

It is *valid* when $\text{assign}(a)$ fits τ for every packet type a . It is *stateless* because every occurrence of type a uses exactly the same annotated path: the same child choices and the same rank at every level. The two schedulers compared below may have different topologies.

An operation is either **push**(a) or **pop**. A run starts from the empty tree and counter zero. The j -th push increments the counter to j and uses j as the arrival stamp. It emits no observation. Each pop contributes exactly one element of $\text{Option}(A)$: **some**(a) on success and **none** on failure. A failed pop leaves both the tree and arrival counter unchanged. Definedness is therefore observable.

The observation records a packet's type, not its source-push identity. In the stateless model these observations contain the same ordering information. Every two occurrences of one type carry equal ranks in every queue they share, so FIFO tie breaking releases them in their source arrival order. Hence the i -th emitted occurrence of type a is the i -th pushed occurrence of that type. Relative to a fixed input operation word, the type trace therefore determines the source-occurrence identity trace.

For a word $w = a_1 \cdots a_n \in A^*$, define its flush operation word by

$$\text{flushOps}(w) = \text{push}(a_1) \cdots \text{push}(a_n) \text{pop}^n. \quad (2)$$

Exactly n pops suffice: fewer reveal a prefix of the full deterministic drain, while additional pops append only **none**. Write $\text{run}(S, o)$ for the list of pop observations produced by operation word o .

Definition 2.1 (The two equivalences). For valid schedulers S, T on A ,

$$\begin{aligned} S \simeq_{\text{flush}} T &\iff \forall w \in A^*. \text{run}(S, \text{flushOps}(w)) = \text{run}(T, \text{flushOps}(w)), \\ S \simeq_{\text{int}} T &\iff \forall o \in \text{Op}(A)^*. \text{run}(S, o) = \text{run}(T, o). \end{aligned}$$

The frozen proposition is the following statement, uniformly over the finite alphabet size.

Theorem 2.2 (Flushes suffice for stateless PIFO trees). *For every $k \in \mathbb{N}$ and all valid stateless schedulers S, T over $\text{Fin}(k)$,*

$$S \simeq_{\text{flush}} T \implies S \simeq_{\text{int}} T.$$

Consequently flush and interleaved equivalence coincide on valid stateless PIFO trees.

The implication from right to left in the final sentence follows immediately because every $\text{flushOps}(w)$ is an operation word. The nontrivial implication is proved in Section 3.

2.4 Correspondence with the frozen Lean statement

Table 1 maps the paper notation to the statement’s definitions. The source file is statement-only: its final item is a definition of a proposition, not a proof of that proposition.

Paper object	Lean name	Source lines
Entry and its three fields	<code>Entry; .val, .rank, .arr</code>	53–57
PIFO contents	<code>Queue</code>	59–60
Strict entry comparison	<code>better</code>	64–66
PIFO pop	<code>qpop</code>	68–77
Tree shape	<code>Topology</code>	81–86
Runtime tree state	<code>Tree</code>	88–94
Empty tree and forest	<code>emptyTree; emptyForest</code>	96–104
Annotated path	<code>Path</code>	106–113
Path well-formedness	<code>pathOk; pathOkAt</code>	115–126
Tree push and list helper	<code>treePush; treePushAt</code>	128–143
Atomic tree pop and list helper	<code>treePop; treePopAt</code>	145–182
Stateless scheduler	<code>Scheduler</code>	186–193
Topology and fixed assignment	<code>Scheduler.topo; Scheduler.assign</code>	191–192
Validity	<code>Scheduler.Valid</code>	194–197
Push/pop operations	<code>Op</code>	216–220
Execution from a state	<code>runFrom</code>	222–235
Execution from the empty state	<code>run</code>	237–240
Flush experiment	<code>flushOps</code>	244–250
Flush equivalence	<code>flushEquiv</code>	252–254
Interleaved equivalence	<code>interEquiv</code>	256–261
Main proposition	<code>statelessFlushImpliesInterleaved</code>	263–269

Table 1: Mathematical-to-Lean correspondence for the frozen formal model.

3 Flushes determine every interleaving

We prove Theorem 2.2. It is convenient to work over an arbitrary finite set A ; transporting schedulers and operation words along a bijection $A \cong \text{Fin}(k)$ gives the frozen statement.

For $B \subseteq A$, write $w|_B$ for the subsequence of a word w whose letters lie in B . A natural-valued rank function induces a total preorder. For such a preorder r , $\text{sort}_r(w)$ is the stable sort of the *occurrences* in w by $(r(a), \text{source arrival})$. For a partition P of A , $\pi_P(w)$ replaces each letter by its P -block. If M is a scheduler, $F_M : A^* \rightarrow A^*$ denotes its flush transformer, with the `some` constructors erased. Validity ensures that a flush of n pushes has n successful pops.

3.1 Operational facts

The following elementary invariant connects the atomic formal semantics with the queue-level reasoning used later.

Lemma 3.1 (Reference balance). *At every valid reachable state, for every internal node and each of its children c ,*

$$\#\{\text{queued references to } c\} = \#\{\text{pending packets in the subtree of } c\}. \quad (3)$$

Consequently a root pop fails exactly when the total pending-packet count is zero; after an internal queue selects a child, recursive failure is impossible.

Proof. Initially both sides of Equation 3 are zero at every node. A valid push through child c appends exactly one reference to c and exactly one packet somewhere below c ; it does not affect the equality for another child. A successful pop selecting c removes one such reference and, recursively, one packet below c . Induction down the selected path gives a successful leaf pop. Hence a positive root count supplies a complete root-to-leaf pop, while count zero means the root queue is empty. The atomic partial-failure branch exists in the definition but is unreachable from a valid reachable state. $\square$

In particular, for a fixed operation word, the **none/some** pattern depends only on the number of preceding pushes and successful pops, not on ranks or topology. Two valid schedulers therefore have identical definedness patterns.

Lemma 3.2 (Timestamp compression). *Replace the arrival stamps occurring in any embedded queue computation by $1, 2, \dots$ in their existing strict order. Every queue selection, and hence every emitted value, is unchanged.*

Proof. Ranks are unchanged. Equal-rank entries are compared only by the order of their arrival stamps, which a strictly increasing replacement preserves. Every choice of a least (rank, arr) key is therefore the same. Induction over the operation word gives the claim. $\square$

This lemma allows a child invoked at sparse global timestamps to be compared with a standalone run whose pushes are consecutively stamped. We will also attach a ghost source-occurrence tag to every entry when couplings need to identify an insertion. The tag is not part of the model and is never observed; erasing it commutes with every transition because queue ordering uses only rank and arrival. Within every reachable queue, and within every abstract pending-occurrence list used below, arrival stamps are pairwise distinct. The same source push may deliberately occur at several selector levels with the same stamp. Consequently (rank, arr) is a strict key within each selector: every selection has a unique minimum. Arrival injectivity is used in the colored-PIFO coupling, while the strict-key consequence is used in the diamond and collapse arguments.

3.2 What two types reveal

Lemma 3.3 (Two-type reduction). *For distinct types x, y , the restriction of a stateless PIFO tree to operation words over $\{x, y\}$ behaves as one stable PIFO with an effective comparison*

$$e(x, y) \in \{x < y, x \sim_e y, y < x\},$$

where $\sim_e$ denotes a rank tie rather than equality of types. The comparison is determined by the two binary flushes in Table 2.

Proof. Follow the assigned paths of x and y together. Stop at their first internal node with different child indices, or at their common leaf if their indices never differ. At every earlier node all relevant references carry the same child value. Such a node forwards one anonymous service for each successful pop and cannot choose between the two types.

At a decisive internal node, its selected child identifies x or y , and the selected child's restricted subtree is monochromatic. At a decisive leaf, the queue emits x or y directly. In either case one rank comparison at this node, with FIFO order on equality, determines every restricted operation word. If x is strictly earlier, both two-letter arrival orders drain as xy ; if the ranks tie, each arrival order is retained; and if y is strictly earlier, both drain as yx . These are exactly the three rows of Table 2. $\square$

effective comparison $e(x, y)$	$F(xy)$	$F(yx)$
$x < y$	xy	xy
$x \sim_e y$	xy	yx
$y < x$	yx	yx

Table 2: Recovery of the effective binary comparison. Both arrival orders are needed to distinguish a rank tie from either strict order.

Notice that $e(x, y)$ is an extensional fact about the restricted scheduler; it need not be read from a particular syntactic node. Flush-equivalent schedulers have the same e for every pair by Table 2.

3.3 A colored-PIFO coupling

The induction will replace a root preorder while retaining only the sequence of selected children. The next lemma justifies that replacement for arbitrary interleavings, including exact cross-color rank ties.

Lemma 3.4 (Colored PIFOs). *Let p, q be total preorders on a finite set of types, and let γ color those types. Suppose*

$$\gamma(x) \neq \gamma(y) \implies \text{cmp}_p(x, y) = \text{cmp}_q(x, y), \quad (4)$$

where comparison has the three values less, equal, and greater. PIFO queues ordered by p and q , subjected to identical pushes and pops, emit identical color traces.

Proof. Refine each color into *cross-profile classes*. Define

$$x \sim y \iff \gamma(x) = \gamma(y) \wedge \forall z \text{ of another color. } \text{cmp}_p(x, z) = \text{cmp}_p(y, z). \quad (5)$$

By Equation 4, the same classes are obtained from q . We use three consequences of totality and transitivity.

First, two distinct classes of one color are strictly separated in the same direction by both preorders. Indeed, an outside type witnessing the difference between their profiles lies weakly on different sides of them; transitivity rules out reversing that separation. Second, if a class ties a type of another color, it is *value-pinned*: every member ties that witness and therefore all members tie one another, under both p and q . Third, comparisons between differently colored occurrences agree outright by hypothesis.

Couple the two queues while maintaining the following invariant:

1. every profile class has the same pending count on both sides; and
2. every class that ties another class has literally the same pending list of ghost-tagged occurrences on both sides.

The invariant holds initially and identical pushes extend both copies in the same class.

Consider a pop. Suppose the p -queue selects occurrence o_1 in class K_1 , while the q -queue selects o_2 in a different class K_2 . Equal class counts provide a pending K_2 -occurrence on the p -side and a pending K_1 -occurrence on the q -side. Minimality of the selected entries implies, member-wise,

$$K_1 \leq_p K_2 \quad \text{and} \quad K_2 \leq_q K_1.$$

The classes cannot have one color: their strict, same-direction separation would contradict these inequalities. They have different colors, so their constant cross-class comparison is common to p and q . The two inequalities force that comparison to be a tie in both directions.

Both classes are therefore value-pinned. By the invariant, each class's pending occurrence list is identical across the queues. Thus o_1 and o_2 are pending in both queues, and their rank components tie within each queue. Minimality on the p -side gives $\text{arr}(o_1) \leq \text{arr}(o_2)$; minimality on the q -side gives the reverse, so the stamps are equal. Both occurrences lie in each queue, where arrival stamps are injective, hence $o_1 = o_2$, contradicting $K_1 \neq K_2$.

Both pops therefore select one profile class and emit the same color. Its pending count decreases on both sides. If the class ties another class, it is value-pinned, so both queues remove its arrival-least pending occurrence—the same occurrence—and the identical-list clause survives. Other lists are untouched. Empty pops agree because total pending counts agree. Induction on the common operation word completes the coupling. $\square$

3.4 Proper flush decompositions

For induction on the alphabet, we expose the first queue that makes a nontrivial routing decision.

Lemma 3.5 (Proper top decomposition). *If $|A| > 1$, every valid stateless scheduler is online-equivalent to a scheduler of the form*

$$M = (P, r, (M_B)_{B \in P}), \quad (6)$$

where P is a proper partition of A into active root children, r is the total preorder induced by root ranks, and M_B is the child scheduler restricted to types in B . Its flush transformer satisfies

$$F_M(w)|_B = F_{M_B}(w|_B) = F_M(w|_B), \quad (7)$$

$$\pi_P(F_M(w)) = \pi_P(\text{sort}_r(w)). \quad (8)$$

Proof. Starting at the root, follow the unique active child used by all types for as long as one exists. Each skipped queue contains only that child value and therefore forwards exactly one service per successful pop; removing such a node preserves every online trace. The walk ends either at the first node with at least two active children or at a leaf. In the former case, partition types by the chosen child. Every block is nonempty and, because at least two are active, proper.

In the leaf case, replace the leaf by an internal root having the old leaf rank for each type and one FIFO singleton child per type. The new root queue mirrors the old leaf queue occurrence-for-occurrence, while a singleton child has no choice to make. Since $|A| > 1$, the singleton partition is proper. This establishes Equation 6 without changing online behavior.

During a flush, child B receives all pushes in $w|_B$ before it receives service, and the root calls it exactly $|w|_B|$ times. Its complete output is therefore $F_{M_B}(w|_B)$, proving the first equality of Equation 7. On a word confined to B , all root references select B , which proves the second. Finally the root drains its reference occurrences by the stable (r, arr) order. Replacing their source types by their child blocks yields Equation 8. $\square$

We call (P, r) a *decomposition* of a word transformer F when Equation 7 and Equation 8 hold. Two consequences will be used without further comment:

$$F(v) = F_{M_B}(v) \quad \text{if every letter of } v \text{ lies in } B, \quad (9)$$

$$F(w)|_B = F(w|_B) \quad \text{for arbitrary } w. \quad (10)$$

The first is *block autonomy*; the second says that deleting other blocks commutes with the transformer.

3.5 Combining two decompositions

Suppose (P, r) and (Q, s) decompose one common flush transformer F . Their meet partition is the set of nonempty incidence cells

$$R = P \wedge Q = \{B \cap C \neq \emptyset : B \in P, C \in Q\}. \quad (11)$$

We will construct a total preorder that agrees with r across P -blocks and with s across Q -blocks. We do *not* need to claim that the resulting pair (R, t) itself decomposes F .

First, every cell inherits autonomy. For $D = B \cap C$, repeated use of Equation 10 gives

$$\begin{aligned} F(w)|_D &= (F(w)|_B)|_C = F(w|_B)|_C \\ &= F((w|_B)|_C) = F(w|_D). \end{aligned} \quad (12)$$

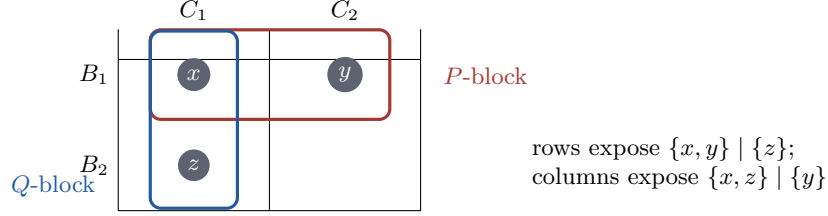


Figure 2: The only nontrivial three-cell incidence shape. Three meet cells form an L when neither all P -coordinates nor all Q -coordinates are distinct. Its two visible decompositions form the diamond used below.

For types in distinct R -cells, prescribe the effective comparison $e(x, y)$ recovered by Lemma 3.3. If P separates the pair, its two-type restriction is chosen at the P -root, and therefore

$$e(x, y) = \text{cmp}_r(x, y). \quad (13)$$

Likewise, if Q separates it,

$$e(x, y) = \text{cmp}_s(x, y). \quad (14)$$

The prescriptions agree when both partitions separate the pair because both are recovered from the same two flushes of F .

Lemma 3.6 (Three-cell diamond). *On one representative type from each of any three distinct meet cells, the pairwise comparisons e form a total preorder.*

Proof. View the cells as occupied positions in the $P \times Q$ incidence grid. If the three P -coordinates are distinct, all comparisons come from the genuine preorder r by Equation 13. If their Q -coordinates are distinct, they all come from s by Equation 14. Otherwise the three positions form an L as in Figure 2; naming its corner x , the restricted transformer has both decompositions

$$\{x, y\} \mid \{z\}, \quad \{x, z\} \mid \{y\}. \quad (15)$$

Assume that the non-strict relation induced by e is not transitive. Since each pair is totally compared, there are distinct a, b, c with

$$a \leq_e b, \quad b \leq_e c, \quad c <_e a. \quad (16)$$

Push one occurrence of each in the cycle order a, b, c , then flush. The arrival stamps are distinct, so every queue computation has a unique least (rank, arr) key. The cycle inequalities determine every strict or weak root comparison, and the fixed arrival order resolves any remaining rank tie; no separate tie case split is needed.

Consider in full the L orientation whose grouped pairs are $\{b, c\}$ and $\{c, a\}$. In the decomposition $\{b, c\} \mid \{a\}$, separated-pair comparisons give root ranks $\rho_a \leq \rho_b$ and $\rho_c < \rho_a$. Thus $\rho_c < \rho_b$, and the root drains references in the order c, a, b : if $\rho_a = \rho_b$, arrival puts a first. The resulting child slots are

$$\{b, c\}, \quad a, \quad \{b, c\}.$$

Within the pair child, b arrived before c and $b \leq_e c$, so the binary recovery table says that its output is b, c . The complete output is therefore b, a, c .

In the decomposition $\{c, a\} \mid \{b\}$, separated comparisons give $\sigma_a \leq \sigma_b \leq \sigma_c$. With arrival order a, b, c , the root drains references in that same order, producing slots

$$\{c, a\}, \quad b, \quad \{c, a\}.$$

The pair child outputs c, a , since $c <_e a$, and the complete output is c, b, a . The first letters differ, contradicting that both decompositions compute the same $F(abc)$.

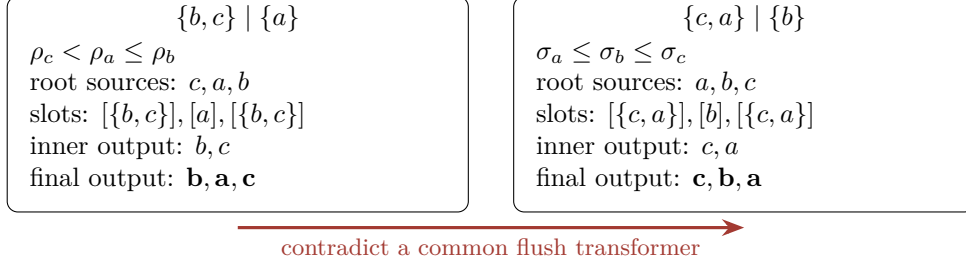


Figure 3: The representative diamond computation for $a \leq_e b \leq_e c <_e a$, using one common arrival order a, b, c . Internal references choose child slots; the selected pair child may emit a different source occurrence.

Cyclic renaming covers the other two placements of the two grouped edges in the comparison cycle. Exchanging the roles of the two decompositions covers the three reflected L orientations. These exhaust the incidence shape, so no failure of transitivity is possible. Reflexivity and totality are already built into the pair comparisons; hence the restriction is a total preorder. $\square$

The diamond controls triples of cells. The next argument shows that no higher-arity obstruction remains.

Lemma 3.7 (Meet-compatible preorder). *If (P, r) and (Q, s) decompose F , there is a natural-valued total preorder t such that*

$$\begin{aligned} \text{cmp}_t(x, y) &= \text{cmp}_r(x, y) && \text{whenever } x, y \text{ lie in distinct } P\text{-blocks,} \\ \text{cmp}_t(x, y) &= \text{cmp}_s(x, y) && \text{whenever } x, y \text{ lie in distinct } Q\text{-blocks.} \end{aligned} \quad (17)$$

Proof. Build a directed constraint graph on the types. Between distinct meet cells, each relation $x \leq_e y$ contributes a weak edge $x \rightarrow y$, marked exactly when $x <_e y$. Thus a preorder tie contributes unmarked edges in both directions. It is enough to prove that no directed cycle contains a marked edge.

Suppose otherwise. Choose a marked edge $a \rightarrow b$ as the head of such a cycle and argue by strong induction on its length. Let c, d be the next vertices after b when present. Only the three fixed chord positions (a, c) , (b, d) , and (a, d) are needed; they are shown in Figure 4.

For length two, the return edge $b \rightarrow a$ contradicts the strict comparison on the marked head. For length three, the consecutive edges join three distinct cells, and Lemma 3.6 supplies a genuine total preorder on the triple; such a preorder cannot contain a marked directed triangle.

Consider a length-four cycle (a, b, c, d) . If a and c occupy distinct cells, compare that chord. An edge $c \rightarrow a$ closes the marked triangle $a \rightarrow b \rightarrow c \rightarrow a$. Otherwise totality gives $a \rightarrow c$, and it is marked: in the three-cell preorder, $a <_e b \leq_e c$ implies $a <_e c$. The cycle $a \rightarrow c \rightarrow d \rightarrow a$ is then a marked triangle. Likewise, if b, d occupy distinct cells, an edge $b \rightarrow d$ yields the marked triangle $a \rightarrow b \rightarrow d \rightarrow a$, which retains the original marked head; if that edge is unavailable, totality makes $d \rightarrow b$ strict, yielding the marked triangle $b \rightarrow c \rightarrow d \rightarrow b$. If both diagonals are intra-cell, the cycle alternates between two meet cells. All four cross-cell comparisons then come from one genuine preorder: from r when P separates those cells, and otherwise from s , since Q must separate them. Its ranks would have to satisfy

$$\rho_a \leq \rho_b \leq \rho_c \leq \rho_d \leq \rho_a \quad \text{while} \quad \rho_b \leq \rho_a \text{ is false,}$$

which is impossible.

Now let the length be at least five. If a, c lie in distinct cells, an edge $c \rightarrow a$ closes the marked triangle through a, b, c . Otherwise the forward chord $a \rightarrow c$ is strict by the same three-cell argument, and replacing $a \rightarrow b \rightarrow c$ by this marked head shortens the cycle. Strong induction excludes it.

We may therefore assume a, c are in one cell. If b, d are in distinct cells and the forward edge $b \rightarrow d$ exists, replacing $b \rightarrow c \rightarrow d$ by it shortens the cycle while retaining the original marked head $a \rightarrow b$. If the forward edge is unavailable, totality makes the reverse edge $d \rightarrow b$ strict, closing a marked triangle through b, c, d . Thus b, d must also be in one cell.

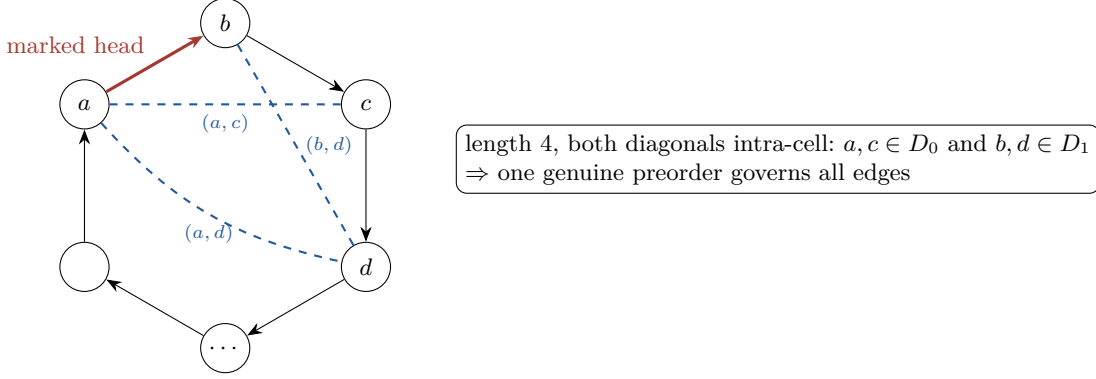


Figure 4: The fixed-head-chord induction. A longer marked cycle is shortened or reduced to a three-cell diamond using only the displayed chord positions; the alternating length-four case is controlled by one original root preorder.

Now a, d lie in distinct cells: otherwise a, c, d would share a cell, contradicting the cross-cell edge $c \rightarrow d$. If the chord $d \rightarrow a$ exists, it closes the already excluded length-four marked cycle $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$. In the other orientation, all four cross-cell comparisons come from one original root preorder because a, c share one cell and b, d share another. In that preorder, $a <_e b \leq_e c \leq_e d$, so $a \rightarrow d$ is strict. Replacing the three-edge prefix by this marked head produces a shorter marked cycle. Strong induction excludes every case, so no marked cycle exists.

It remains to realize all constraints by natural ranks. Write $z \rightsquigarrow x$ for reflexive graph reachability, allowing a path of length zero, and set

$$t(x) = \#\{z : z \rightsquigarrow x \wedge x \not\rightsquigarrow z\}. \quad (18)$$

Let $L_x = \{z : z \rightsquigarrow x \wedge x \not\rightsquigarrow z\}$. If $x \rightarrow y$ is a weak edge and $z \in L_x$, then $z \rightsquigarrow x \rightarrow y$. A return path $y \rightsquigarrow z$ would imply $x \rightsquigarrow z$, a contradiction; hence $z \in L_y$, and $L_x \subseteq L_y$. If $x \rightarrow y$ is marked, no path returns from y to x , since it would complete a marked cycle. Thus $x \in L_y$, whereas reflexivity gives $x \notin L_x$. The inclusion is strict and finiteness gives $t(x) < t(y)$. A prescribed tie contributes weak edges both ways, so the two inclusions give equal predecessor sets and equal ranks. Thus t realizes every prescribed e -comparison. The constraints in Equation 13 and Equation 14 give exactly Equation 17. $\square$

3.6 Strong induction: swap, refine, and collapse

We now prove Theorem 2.2 simultaneously for every finite alphabet by strong induction on $|A|$. If A is empty, every operation is a failed pop. If $|A| = 1$, each successful pop emits the sole type, and Lemma 3.1 says that success is determined only by the push/pop count.

Assume $|A| > 1$, and let S, T be flush-equivalent valid stateless schedulers. By Lemma 3.5, normalize them online to proper top decompositions

$$S = (P, r, (S_B)_{B \in P}), \quad T = (Q, s, (T_C)_{C \in Q}), \quad (19)$$

with common flush transformer F . Form $R = P \wedge Q$ and choose the meet-compatible preorder t from Lemma 3.7.

Swap before refining. Replace S 's root order r by t , leaving its partition and children unchanged. Across distinct P -blocks the two root preorders agree by Equation 17. Apply Lemma 3.4 with colors P . The root has the same complete online P -token trace, so every child receives the same driven word. Thus

$$S \simeq_{\text{int}} S^t \quad \text{and} \quad F_{S^t} = F. \quad (20)$$

Symmetrically $T \simeq_{\text{int}} T^t$. Applying Equation 8 directly to the two swapped schedulers gives both pattern facts, for every w ,

$$\pi_P(F(w)) = \pi_P(\text{sort}_t(w)), \quad \pi_Q(F(w)) = \pi_Q(\text{sort}_t(w)). \quad (21)$$

This order of operations is important: after the swaps, no separate meet-cell control-word construction is needed.

Refine one proper block. For every meet cell $D \in R$, choose a valid scheduler U_D realizing the restricted transformer F on D . For example, restrict S 's fixed assignment to types in D ; cell autonomy Equation 12 identifies its flush transformer with $F|_D$.

Fix $B \in P$. Let V_B have a t -ordered root routing to the nonempty cells $D = B \cap C$ below B , with child U_D at cell D . We claim

$$F_{S_B} = F_{V_B}. \quad (22)$$

Take a word v over B . A word over B is uniquely reconstructed from its Q -block pattern and its subsequence in every Q -block. By block autonomy, $F_{S_B}(v) = F(v)$, whose Q -pattern is $\pi_Q(\text{sort}_t(v))$ by Equation 21. The root-pattern equation for V_B gives the same pattern.

Fix a block $C \in Q$ and set $D = B \cap C$. If $D = \emptyset$, both C -subsequences are empty because both output words lie in B . If $D \neq \emptyset$, then $D \in R$, so U_D is defined. Because v is a B -word, $v|_C = v|_D$, and

$$\begin{aligned} F(v)|_C &= F(v|_C) = F(v|_D) \\ &= F_{U_D}(v|_D) = F_{V_B}(v)|_C. \end{aligned} \quad (23)$$

Here the first equality is autonomy for the Q -decomposition, the third is the choice of U_D , and the last is Equation 7 for V_B ; filtering by C and by D agrees because every output of V_B lies in B . Equal pattern and equal subsequences prove Equation 22.

The partition P is proper, so $|B| < |A|$. The induction hypothesis applied to Equation 22 gives

$$S_B \simeq_{\text{int}} V_B. \quad (24)$$

Substitute V_B for every child S_B under the already swapped t -root. The root reference queue is unchanged. Child B receives its projected pushes and one anonymous pop whenever a B -token is selected, so the driven operation word is the same on both sides. Its global arrival stamps may have gaps; Lemma 3.2 converts them monotonically to the standalone instance of Equation 24. Child substitution therefore preserves the entire online trace.

Collapse equal selector levels. After substitution, the scheduler has two routing levels: an outer root over the P -blocks and, inside each B , a root over its R -cells. Both levels use preorder t . Let U instead have one t -root routing directly to the same children U_D .

Couple these schedulers with a set Ω of pending ghost-tagged selector-reference occurrences. Maintain:

1. the two-level outer queue is Ω , labeled by P -block;
2. U 's root queue is Ω , labeled by R -cell;
3. the inner queue below B is $\Omega|_B$, labeled by R -cell; and
4. corresponding U_D states are identical.

Pushes extend exactly the corresponding queues and child state. On a pop, let the outer queue select occurrence $o \in B$. Every selector uses the same key (t, arr) , and arrival stamps within Ω are pairwise distinct, so o is the unique minimum of Ω and therefore also of $\Omega|_B$. The inner queue selects the same occurrence; the one-level root, holding the same Ω with the same keys, also selects o . There is no full-key tie case. Both sides call the same U_D state. A successful lower pop removes o from all corresponding selectors. An atomic lower failure rolls both sides back identically. The invariant survives, and

$$S \simeq_{\text{int}} U. \quad (25)$$

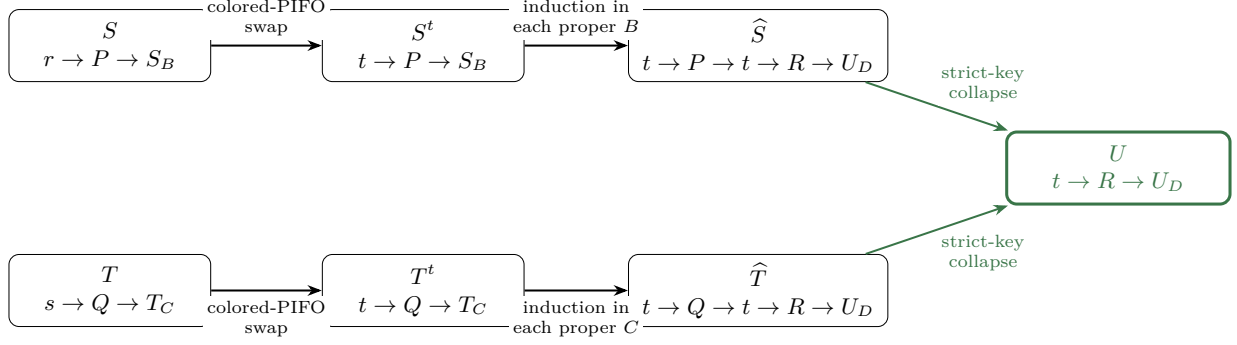


Figure 5: Proof architecture. The common preorder t is installed before the children are refined. Strong induction produces the same meet-cell children on both sides, and identical selector levels collapse to one common scheduler U .

Starting from T , swap its root to t , refine each proper $C \in Q$ into a t -root over the same meet-cell schedulers U_D , and collapse. The proof of block flush equality is the mirror image of Equation 22: use the P -pattern in Equation 21 and autonomy from the S -side. It gives

$$T \simeq_{\text{int}} U. \quad (26)$$

Transitivity of equality of run outputs yields $S \simeq_{\text{int}} T$. The argument has coupled successful outputs; Lemma 3.1 restores the identical `none` positions for over-pops. This proves Theorem 2.2.

Why the induction is well founded. Only the original blocks $B \in P$ and $C \in Q$ are recursive arguments, and both top partitions are proper. A meet cell may equal an entire original block, and V_B may consequently have just one active cell, but the measure is still $|B| < |A|$, not tree height or the size of an introduced child. Repeated packet types cause no problem: every projection, stable sort, and reconstruction above is a computation on occurrences.

4 A stateful separation

The fixed type-to-path assignment in Theorem 2.2 is essential. We now work through two flat three-leaf trees whose assignment may inspect the persistent arrival counter. They agree on every flush word but disagree on a five-operation interleaving.

For this example only, let a transaction for push number j choose a triple

$$\text{tx}(j) = (\text{leaf}, \text{leaf rank}, \text{root rank}).$$

The choice is independent of the packet label but depends on j . A pop does not reset the arrival counter. In Table 3, notation $C(1, 1)$ means leaf C , leaf rank 1, root rank 1.

push number	scheduler S_1	scheduler S_2
1	$C(1, 1)$	$E(1, 3)$
2	$B(1, 3)$	$E(2, 1)$
3	$C(2, 2)$	$F(1, 2)$
$j \geq 4$	$G(j, 100 + j)$	$G(j, 100 + j)$

Table 3: Arrival-dependent assignments for the counterexample. Leaf names are local to a scheduler; both trees have three leaves.



Figure 6: The three-push flush. A root entry selects a child queue, not its source packet. The E_2 reference can therefore release occurrence 1, allowing the two different trees to realize the same permutation.

Proposition 4.1 (Stateful flush/interleaving separation). *The schedulers in Table 3 have identical flush output for every finite packet word. They are not interleaved-equivalent.*

Proof. First label packet occurrences by their source arrival numbers. After pushes 1, 2, 3, scheduler S_1 's root priority order is

$$C_1, C_3, B_2,$$

because the respective root ranks are 1, 2, 3. Leaf C contains occurrences 1, 3 at leaf ranks 1, 2, while B contains occurrence 2. The drain is therefore 1, 3, 2.

Scheduler S_2 's root priority order is instead

$$E_2, F_3, E_1.$$

Leaf E contains occurrence 1 at rank 1 and occurrence 2 at rank 2; F contains occurrence 3. The first root reference, E_2 , selects leaf E , but that leaf emits occurrence 1, not the occurrence that created the root reference. Then F emits 3, and the last E -service emits 2. The drain is again 1, 3, 2. This is the indirection visible in Figure 6.

For $j \geq 4$, both schedulers insert into the common tail leaf G , with root ranks $100 + j$ and leaf ranks j . These entries follow the first three and are ordered increasingly, so both append $4, 5, \dots, n$. Directly checking the shorter prefixes gives

n	identity drain
0	ϵ
1	1
2	1, 2
$n \geq 3$	1, 3, 2, 4, 5, $\dots, n$.

(27)

Now take an arbitrary label word $x_1 \dots x_n$. The transactions ignore labels, so replacing each identity in Equation 27 by its source label shows that both schedulers return $x_1 x_3 x_2 x_4 \dots x_n$, with the displayed short cases. Thus they agree on every flush word, not merely on words with distinct labels.

For the interleaving

$$\text{push; pop; push; push; pop,} \tag{28}$$

the first push/pop emits occurrence 1 and empties both trees, but leaves the counter at one. Scheduler S_1 then inserts occurrence 2 into B at root rank 3 and occurrence 3 into C at root rank 2, so the final pop emits 3. Scheduler S_2 inserts occurrence 2 into E at root rank 1 and occurrence 3 into F at root rank 2. The old rank-1 packet in E is gone; the selected E leaf therefore emits occurrence 2. The identity traces are 1, 3 and 1, 2.

This is observable with a two-type alphabet. Choose labels $x_1 = 0, x_2 = 0, x_3 = 1$. The operation word $\text{push}(0); \text{pop}; \text{push}(0); \text{push}(1); \text{pop}$ produces type traces 0, 1 for S_1 and 0, 0 for S_2 . Hence interleaved equivalence fails. $\square$

The artifact's executable model stores packet identity as the arrival number, which is a stronger observation than type alone. Lean kernel-checks the separating interleaving and the flush equations through $n = 8$

in `Pifo.lean` using `decide`; a separate `native_decide` theorem in `Extended.lean` checks them through $n = 30$. The universal flush claim in Proposition 4.1 is established by the symbolic calculation Equation 27; the artifact explicitly does not contain an all- n induction. This boundary is recorded again in Section 12.

The standalone checker records only successful pop outputs, whereas the frozen model records a failed pop as `none`. Instrumenting it with observable failures embeds the example in the paper’s observation convention: every pop in every flush and in the separating trace succeeds, so none of the displayed outputs changes.

5 What a stateless PIFO tree is

The proof suggests an extensional reading that is simpler than the concrete queue state. A stateless PIFO tree is an *arbitration hierarchy*. Each internal PIFO arbitrates among child colors; its selected reference grants one service to a child, whose own queue decides which packet that service releases. The parent reference and the emitted packet need not come from the same push. This distinction is not an implementation corner case: it drives both the three-cell diamond and the stateful example.

Pairwise orders live at first divergence. For two types, all nodes above their first different child choice are merely forwarders. Their effective order is the rank comparison at that first divergence, or at their common leaf if they never diverge. In tree language, this is the decisive queue at the pair’s lowest shared routing point. The binary recovery table observes exactly that comparison, including equality. Thus a complicated topology has a simple two-type projection.

Pairwise comparisons do not mean that an entire tree is one flat priority queue. Different pairs may be decided at different queues, and a parent’s choice names only a subtree. Inside the chosen subtree, another type may be released first. The meet proof is precisely the argument that the cross-block comparisons needed to reconcile *two* flush-equivalent hierarchies have no cyclic obstruction. It does not claim that an arbitrary scheduler’s entire semantics can be read from an unconstrained global table of pair comparisons.

Partitions are structural witnesses, not observables. The proof takes the top partitions P and Q from the two supplied trees. It never reconstructs either partition from the common flush transformer F . This matters when strict comparisons happen to align across a hidden boundary, or when tied blocks can be merged without affecting a drain. The construction is pair-dependent: it refines the two known partitions to $P \wedge Q$ and builds only the common preorder required for this pair of schedulers.

Depth is not itself observable. The phrase requires care. Arbitrary changes to a tree’s hierarchy can change behavior, but several forms of depth have no extensional content:

- a chain of nodes through which every active type chooses one child is an online-inert forwarding chain and can be contracted;
- a terminal leaf can be expanded into a root with the leaf’s ranks and FIFO singleton children, occurrence-for-occurrence; and
- two consecutive selector levels over the same type preorder collapse to one level because all three relevant queues choose the same unique (rank, arr)-minimum.

Accordingly the main induction is on alphabet size, not topology height. The tree supplies a recursive partition of types; raw distance from the root is not part of the observation.

Why statelessness changes the observational theory. A fixed annotated path gives every occurrence of one type the same ranks and routing choices. Same-type occurrences therefore retain source order, so a type trace also determines the identity trace. More importantly, restricting to a block or cell yields another scheduler on a smaller alphabet, suitable for induction. With a stateful assigner, two occurrences carrying the same label can be sent to different leaves or assigned different ranks. A prefix pop can then remove the

packet that a future root reference would have caused a child to release. The counterexample exploits this residual-state change.

The result should therefore be read as a class theorem. Stateless PIFO trees admit no flush-equivalent but interleaving-distinct pair; once arrival- or state-dependent assignment is admitted, such pairs exist. This does not say that every stateful scheduler has a distinguishable partner, only that the unqualified implication is false for the enlarged class.

What “testing by flushes” guarantees. Suppose two valid stateless implementations are known to agree on the drain of every finite packet word. The theorem guarantees agreement for mixed command traces, including failed over-pops and arbitrarily sparse arrivals to a child. That implication alone does not provide an effective test. The separate structural analysis in Section 6 and Section 7 supplies a canonical component tree as a complete invariant, an $O(k^3h)$ decision procedure on a unit-cost RAM, and a two-or-three-push suite whose $\Theta(k^3)$ triple-support count is optimal among tests using at most three types. The pair-dependent comparator in Section 8 gives a second decision route that constructs no normal form and returns either a small flush witness or an equivalence proof DAG.

Normalization is semantic, not rewrite irreducibility. The active rank-swap, collapse/regrouping, and leaf-explosion equations are useful proof moves, but their bidirectional relation is nonterminating and a canonical tree can still take a regrouping step. The bottom-up normalizer of Section 9 succeeds by carrying exact restriction-ready pair/triple summaries and reconstructing the canonical tree, not by seeking an irreducible term. The NbE construction of Section 10 gives a different semantic organization: it evaluates to the whole flush transformer and reifies factor partitions, yielding another finite-alphabet proof but no effective factor test. Finally, meet promotion in Section 11 removes alphabet-size induction; its topology-recursive theorem extends the main observational result to arbitrary alphabets one finite operation support at a time, without claiming a global infinite-alphabet normal form.

6 Normalization and canonical forms

The proof of Theorem 2.2 joins two schedulers through a decomposition chosen from both of them. A stronger consequence is that each scheduler has its own canonical decomposition tree. The tree is finite, is recoverable from flush behavior, and completely characterizes interleaved behavior.

Throughout this section $A = \text{Fin}(k)$, with its fixed enumeration, and all schedulers are valid. For $X \subseteq A$, write $S|_X$ for the scheduler obtained by retaining only the types in X . Runs whose pushes lie in X are unchanged by this restriction.

6.1 The local data

For distinct $x, y \in A$, let

$$\text{eff}_S(x, y) \in \{<, =, >\}$$

be the effective comparison of their two-type restriction. Concretely, follow their assigned paths together through every common child. At their first different child, compare the two reference ranks; if they instead end at one leaf, compare their leaf ranks. By Lemma 3.3, this one comparison determines the restricted scheduler on every operation word. It is local under restriction and is recovered from the two drains in Table 2.

Triples carry one further bit. For comparisons $c, d \in \{<, =, >\}$, let $\text{comp}(c, d)$ be their transitive composition: composing with $=$ returns the other comparison, two $<$ ’s compose to $<$, two $>$ ’s compose to $>$, and opposite strict signs give **none**.

Definition 6.1 (Rooted-triple obstruction). For pairwise-distinct x, y, z , restrict S to these three types and use the triple-flattening lemma to replace that restriction by an interleaved-equivalent flat scheduler. Write

$\ell(u)$ for the leaf of type u in this representative. Then $R_S(x, y \mid z)$ holds when

$$\begin{aligned} \ell(x) &= \ell(y) \neq \ell(z), \\ \text{comp}(\text{eff}_S(x, z), \text{eff}_S(z, y)) &= \text{some}(c), \quad c \neq \text{eff}_S(x, y). \end{aligned} \tag{29}$$

The triple lemma makes this predicate independent of the chosen flat representative. It is also local under every further restriction containing x, y, z .

The structural construction of the required flat representative is given in Section 7.1; its $2 + 1$ case is important because the two types in the shared branch must be collapsed to one synthetic leaf.

For $X \subseteq A$, form an undirected graph $G_S(X)$ on X by putting an edge between distinct x, y exactly when some $z \in X \setminus \{x, y\}$ satisfies

$$R_S(x, y \mid z) \vee R_S(y, x \mid z). \tag{30}$$

Let $\Pi_S(X)$ be the partition of X into connected components of this graph.

Lemma 6.2 (Locality and properness). *The data eff_S and R_S on X agree with the corresponding data of $S|_X$. If $|X| > 1$, every block of $\Pi_S(X)$ is a proper subset of X .*

Proof. Pair locality follows by suppressing path prefixes used by both types; triple locality follows from the same common-prefix suppression and the triple-flattening lemma. For properness, descend through every node at which all types in X use one child. If this reaches a leaf, every triple lies in one flat leaf and Equation 29 is false, so the graph has no edges. Otherwise the first branching node partitions X into at least two nonempty child alphabets, and no obstruction edge crosses those alphabets. Thus $G_S(X)$ is disconnected in either case. $\square$

The component partition has exactly the compatibility needed at a canonical root.

Lemma 6.3 (Cell feasibility). *For every $X \subseteq A$ with $|X| > 1$, there is a rank function $q_X : X \rightarrow \mathbb{N}$ such that*

$$\text{cmp}_{q_X}(x, y) = \text{eff}_S(x, y) \quad \text{whenever } x, y \text{ lie in different } \Pi_S(X)\text{-blocks.} \tag{31}$$

There is no constraint on the root comparison of two types in one block.

The lemma is the cell-feasibility part of the top-decomposition argument: one normalizes each child order in turn, while the strong preservation clause keeps all previously treated components fixed. The resulting finite total preorder is represented by natural ranks. The absence of a within-block constraint is essential—that comparison is resolved by the child and is not an observable root invariant.

6.2 The component-decomposition normal form

Definition 6.4 (Abstract canonical tree). Define $\mathcal{N}_X(S)$ by strong recursion on $|X|$.

- If $X = \emptyset$, it is the empty tree, represented by a zero-child node.
- If $X = \{x\}$, it is one leaf carrying x .
- If $|X| > 1$, let $\Pi_S(X) = \{C_1, \dots, C_m\}$. The root is labelled by this partition and by the table

$$\text{eff}_S \mid_{\{(x, y) : [x]_{\Pi_S(X)} \neq [y]_{\Pi_S(X)}\}}, \tag{32}$$

and its children are $\mathcal{N}_{C_i}(S|_{C_i})$, one for each block.

Put $\mathcal{N}(S) = \mathcal{N}_A(S)$. The recursion terminates by Lemma 6.2.

For $k > 0$, a concrete normal form $N(S)$ is any scheduler realizing $\mathcal{N}(S)$: list siblings in a fixed order, use a witness from Lemma 6.3 at each internal node, and continue through the recursively assigned child to the singleton leaf, where the irrelevant leaf rank may be set to zero. Such a scheduler is valid by construction.

The empty alphabet needs a separate concrete convention. Let

$$E_0.\text{topo} = \text{node}[], \quad E_0.\text{assign} : \text{Fin}(0) \rightarrow \text{Path} \text{ the unique empty function,} \quad N(S) = E_0. \quad (33)$$

This is valid vacuously. Since an operation over $\text{Fin}(0)$ cannot push, every operation is a pop and every pop from the initial state returns **none**. Hence all zero-type schedulers are interleaved-equivalent to E_0 .

The concrete representative is canonical only up to inessential presentation. Equality of normal forms below means literal equality of the labelled abstract trees $\mathcal{N}$, or, equivalently, equality of concrete realizations after identifying:

1. a permutation of sibling blocks together with the corresponding remapping of child indices;
2. any root-rank reparametrization preserving all cross-block comparisons, with within-block root ranks left free; and
3. any leaf-rank reparametrization preserving the induced weak order.

Child indices do not break ties, so the first identification is operationally invisible; queues inspect only ranks and arrival stamps.

Theorem 6.5 (Canonical normalization). *For every valid scheduler S ,*

$$S \simeq_{\text{int}} N(S). \quad (34)$$

Proof. The $k = 0$ case was just proved. If $k = 1$, unary active nodes merely forward service, and the lone type is FIFO at its leaf; contracting those nodes and setting the irrelevant leaf rank to zero gives the singleton leaf $N(S)$. For $k > 1$, use strong induction on the support size and suppress every unary active node. If the reduced scheduler is a leaf, expand it to a root ordered by the old leaf ranks and route each type to a singleton child; this preserves every run. Otherwise the canonical top-decomposition lemma replaces the root by one child for each $C \in \Pi_S(A)$, uses a feasible root order satisfying Equation 31, and gives that child the behavior of $S|_C$. Colored root congruence and stable child projection make this replacement online-invisible.

Every C is proper, so the induction hypothesis replaces $S|_C$ by $N(S|_C)$. Interleaved equivalence is a congruence for the node construction: the root couples only the component color, and paired children then emit the same packets. The resulting scheduler is $N(S)$. $\square$

6.3 Flush determination and completeness

Lemma 6.6 (Flush determination). *The pair (eff_S, R_S) , and therefore $\mathcal{N}(S)$, is determined by the flush transformer F_S .*

Proof. For each distinct pair, the two drains of xy and yx recover the exact comparison by Table 2; both arrival orders are needed. For each distinct triple, restrict and relabel its types, flatten the restriction, and use the finite three-label recovery lemma. The drains of the injective words of lengths two and three determine every rooted obstruction bit; the singleton drains are fixed. Thus all of eff_S and R_S are functions of F_S .

At every recursive support X , locality makes $G_S(X)$, and hence $\Pi_S(X)$, a function of the restricted R_S -table. The node label then uses only the corresponding cross-block part of eff_S . Strong recursion therefore reconstructs the whole labelled tree. $\square$

Theorem 6.7 (Canonical characterization). *For valid schedulers S, T over A ,*

$$S \simeq_{\text{int}} T \iff \mathcal{N}(S) = \mathcal{N}(T) \iff (\text{eff}_S = \text{eff}_T \wedge R_S = R_T). \quad (35)$$

In particular, the canonical normal form is determined by flush behavior.

Proof. Equality of the two data tables forces equal canonical trees: locality gives the same graph and component partition on every recursively reached support, and equality of eff gives the same cross-block label. Conversely, if the canonical trees agree, choose one concrete realization N_0 of their common tree. By Theorem 6.5, $S \simeq_{\text{int}} N_0 \simeq_{\text{int}} T$. This implies flush equivalence, so Lemma 6.6 recovers equal data tables. This semantic argument is important: an arbitrary triple need not itself occur as a support in the canonical recursion, so R cannot in general be read directly from one node of the tree.

Finally, interleaved equivalence implies flush equivalence and hence equal data and equal trees. The reverse implication is the common-realization argument above. These implications establish both equivalences in Equation 35. $\square$

7 Deciding equivalence

The canonical characterization turns universal operational equivalence into a finite structural comparison. This section gives the extractor, accounts for its cost, and then identifies an asymptotically optimal finite-support test class.

7.1 Structural extraction

An effective comparison is obtained by scanning two assigned paths. Advance while both paths choose the same child. If they reach a common leaf, compare their leaf ranks; at their first different child, compare their reference ranks at that node. The scan takes time linear in the path length and returns $\text{eff}_S(x, y)$.

The rooted-triple extractor must do more than suppress a common unary prefix. For pairwise-distinct x, y, z , define $\text{TripleDescriptor}(S; x, y, z)$ as follows.

1. Advance the three path cursors while all three select the same child. If they end in one common leaf, return a one-leaf flat descriptor with the three encountered leaf ranks.
2. At the first node where their selected children are not all equal, retain the three reference ranks at that node as synthetic root ranks.
3. For a $1 + 1 + 1$ child split, route the three types to three synthetic singleton leaves.
4. For a $2 + 1$ split, suppose u, v select the shared child. Replace their entire residual subtrees by *one* synthetic shared leaf. Use leaf ranks

$$(r_u, r_v) = \begin{cases} (0, 1), & \text{if } \text{eff}_S(u, v) \text{ is } <, \\ (0, 0), & \text{if } \text{eff}_S(u, v) \text{ is } =, \\ (1, 0), & \text{if } \text{eff}_S(u, v) \text{ is } >. \end{cases} \quad (36)$$

Route the remaining type to another singleton leaf; its leaf rank is immaterial.

Evaluate Equation 29 on this constant-size flat descriptor. The result is $R_S(x, y \mid z)$. Unary transparency justifies the initial scan, the two-type lemma justifies the synthetic shared leaf, and the triple-flattening lemma identifies the descriptor's obstruction bit with that of the original restriction.

Example 7.1 (Why the residual pair must be collapsed). Take topology $\text{node}[\text{node}[\text{leaf}, \text{leaf}], \text{leaf}]$ and paths

	root (child, rank)	inner (child, rank)	leaf rank
x	(0, 0)	(0, 2)	0
y	(0, 2)	(1, 0)	0
z	(1, 1)	–	0

(37)

The first split is $2 + 1$, but x and y end at different terminal leaves. Their residual effective comparison is $>$, so the shared synthetic leaf receives ranks $(1, 0)$, while its flat root receives ranks $(0, 2, 1)$ for x, y, z . Hence

$$\text{eff}(x, z) = <, \quad \text{eff}(z, y) = <, \quad \text{comp}(<, <) = < \neq \text{eff}(x, y),$$

and $R_S(x, y \mid z)$ is true. Directly, $F_S(xyz) = yzx$: the root removes the reference inserted by x , but the selected inner queue emits y . Looking only at terminal-leaf equality would incorrectly return false.

For valid input schedulers S, T , the Boolean decision procedure streams these local invariants.

1. For every unordered pair $\{x, y\}$, compute and cache one orientation of $\text{eff}_S(x, y)$ and $\text{eff}_T(x, y)$; reject on the first mismatch. The reverse orientation is determined automatically.
2. For every ordered pairwise-distinct triple (x, y, z) , compute $R_S(x, y \mid z)$ and $R_T(x, y \mid z)$ with the descriptor above and compare the bits immediately. Reject on a mismatch; do not store the cubic tables.
3. Accept if both loops finish.

For $k = 0$ or $k = 1$, the relevant loops are empty, as required by the base cases of the canonical form.

Theorem 7.2 (Structural decision). *For valid schedulers S, T , the procedure accepts exactly the interleaved-equivalent pairs:*

$$\text{Decide}(S, T) = \text{true} \iff S \simeq_{\text{int}} T. \quad (38)$$

Proof. Acceptance says exactly that the effective-comparison tables and all rooted obstruction bits agree. Apply Theorem 6.7. The two finite loops also prove termination. $\square$

Theorem 7.3 (Decision complexity). *For valid input schedulers S, T , let h be the larger of 1 and the maximum number of constructors in an assigned path of either input, including its terminal leaf; thus $h = 1$ when $k = 0$. On a unit-cost RAM, structural equivalence is decidable in*

$$O(k^3 h) \text{ time} \quad \text{and} \quad O(k^2) \text{ working space} \quad (39)$$

beyond the immutable input.

Proof. There are $\Theta(k^2)$ pair scans, each of length at most h , and their ternary results occupy $O(k^2)$ entries. There are $\Theta(k^3)$ triple scans. A common-prefix scan, together with at most one pair-suffix scan if the cache were absent, visits $O(h)$ path cells; with the cache, Equation 36 is constant time. Each pair of obstruction bits is discarded after comparison, so no cubic table is stored. An iterative cursor implementation uses constant transient space; a recursive one adds an $O(h)$ call stack. $\square$

If ranks and child indices have maximum bit width B , charging comparison by bit complexity gives the conservative time bound $O(k^3 h B)$. The bound in Theorem 7.3 is for the Boolean decision only: it does not hide construction of $\mathcal{N}$ or synthesis of concrete ranks. For comparison, if all R -bits are first stored, a direct decomposition-tree construction scans

$$O\left(k^3 h + \sum_X |X|^3\right) \subseteq O(k^3 h + k^4) \quad (40)$$

over the laminar family of canonical blocks X , using $\Theta(k^3)$ extra storage. There are at most $k - 1$ non-singleton blocks, which gives the displayed $O(k^4)$ summand. Re-extracting rather than storing the bits gives the looser $O(k^4 h)$ bound. No separate complexity claim is made for choosing integer witnesses to cell feasibility.

7.2 Two- and three-push tests suffice

Define the finite suite

$$\mathcal{T}(A) = \{\text{flushOps}(w) : w \text{ is injective and } 2 \leq |w| \leq 3\}. \quad (41)$$

It has

$$k(k - 1) + k(k - 1)(k - 2) = \Theta(k^3) \quad (42)$$

tests, each with at most three pushes and three pops. Singleton drains are fixed by packet conservation and supply no test information.

Theorem 7.4 (Two- and three-push sufficiency). *Suppose valid S, T agree on every test in $\mathcal{T}(A)$. Then $S \simeq_{\text{int}} T$.*

Proof. Fix distinct x, y . The tests xy and yx , after restricting and consistently relabelling the two types, recover $\text{eff}(x, y)$ by Table 2. Thus the two effective tables agree.

Now fix distinct x, y, z . Restrict and relabel these types as $\text{Fin}(3)$. The triple-flattening lemma supplies a valid flat representative for each restriction; this is a semantic replacement, not an assertion that the original restrictions are flat. Their length-two and length-three injective drains agree by hypothesis, and their singleton drains agree automatically. The finite three-label recovery lemma says that these fifteen nonempty injective drains determine the effective table and all labelled obstruction bits of a flat realization. Transporting back gives $R_S(x, y \mid z) = R_T(x, y \mid z)$. Varying the pair and triple gives equal (eff, R) , so Theorem 6.7 applies. When $k < 2$, the empty or singleton canonical base case gives the conclusion. $\square$

Thus repetitions are unnecessary: one occurrence of each of at most three types recovers the complete invariant.

7.3 A cubic support lower bound

Pair data alone is insufficient. The following explicit family will also force every unordered triple support to appear in any universal suite whose tests use at most three types.

Example 7.5 (Equal pair behavior, different triple behavior). On types x, y, z , use the flat topology $\text{node}[\text{leaf}, \text{leaf}, \text{leaf}]$. In type order x, y, z , let ℓ , ρ , and λ denote the leaf, root-rank, and leaf-rank vectors:

$$\begin{array}{c|ccc} & \ell & \rho & \lambda \\ \hline S & (0, 0, 1) & (0, 2, 1) & (1, 0, 0) \\ S' & (0, 1, 1) & (1, 0, 2) & (0, 1, 0) \end{array} \quad (43)$$

The third leaf is initially unused. Both schedulers have

$$(\text{eff}(x, y), \text{eff}(x, z), \text{eff}(z, y)) = (>, <, <). \quad (44)$$

Consequently their restrictions to every pair agree on all operation words. However, $R_S(x, y \mid z)$ is true and $R_{S'}(z, y \mid x)$ is true, giving component partitions $\{\{x, y\}, \{z\}\}$ and $\{\{x\}, \{y, z\}\}$, respectively. The distinction is visible in one flush:

$$F_S(xyz) = yzx, \quad F_{S'}(xyz) = zxy. \quad (45)$$

For example, S 's root first selects the reference inserted by x , but the shared child emits y ; this is the same reference-hijacking phenomenon as in Example 7.1.

Theorem 7.6 (Support-level optimality). *For test suites whose individual tests use at most three packet types, every universal equivalence suite over k types needs at least*

$$\binom{k}{3} = \Omega(k^3) \quad (46)$$

distinct triple supports. Together with Equation 41, the support complexity is $\Theta(k^3)$.

Proof. Embed Example 7.5 on an arbitrary target triple $\{a, b, c\}$. Route every outsider identically in both schedulers to the common unused third leaf, give it root rank 3, and give it leaf rank 0; retain target ranks 0, 1, 2. The choice 3 is load-bearing. Every target-outsider effective comparison is identical, and an outsider cannot obstruct a shared target pair because the two comparisons through it have opposite strict signs. A target likewise cannot obstruct a pair of outsiders.

Therefore the two extended schedulers agree on every restriction omitting at least one of a, b, c , while their target-triple flush still differs. A test using at most three types can cover only one unordered triple, so a universal suite must cover every one of the $\binom{k}{3}$ choices. $\square$

This is a lower bound on supports. It does not say that a particular order of each triple is sufficient, that every pair word is indispensable, or that $\mathcal{T}(A)$ is inclusion-minimal.

7.4 Orientations and shortened drains

The six orders of one triple cannot be replaced by a fixed choice of three. In fact, any selected orientation can be the only distinguishing one.

Example 7.7 (A uniquely distinguishing triple orientation). Consider the following flat realizations, with all vectors in type order 0, 1, 2:

	leaf	root rank	leaf rank
U	(0, 0, 2)	(2, 1, 1)	(2, 2, 2)
V	(0, 1, 0)	(0, 0, 0)	(1, 0, 0)

(47)

Both have effective table $(\text{eff}_{01}, \text{eff}_{02}, \text{eff}_{12}) = (=, >, =)$, while their obstruction-edge tuples (01, 02, 12) are (true, false, false) and (false, true, false). All ordered-pair drains agree:

$$01 \mapsto 01, \quad 02 \mapsto 20, \quad 10 \mapsto 10, \quad 12 \mapsto 12, \quad 20 \mapsto 20, \quad 21 \mapsto 21. \quad (48)$$

Five of the six triple drains also agree; exactly 012 differs:

push order	F_U	F_V
012	021	210
021	201	201
102	120	120
120	120	120
201	201	201
210	210	210

(49)

Proposition 7.8 (No three-orientation subfamily). *For each of the six orders of three fixed types, there are valid flat schedulers that differ on exactly that triple drain while agreeing on the other five and on all ordered-pair drains. Hence no fixed choice of three triple orientations, even together with all pair drains, is universally sufficient.*

Proof. Relabelling types by a permutation relabels every successful output, by an induction on run. It can therefore move the sole differing word 012 in Example 7.7 to any chosen orientation while preserving agreement on the other five and on every pair observation. $\square$

Thus each orientation is individually indispensable when the other five and all pair observations are retained. This is a statement about the triple-orientation subfamily, not inclusion-minimality of a mixed global suite.

One observation in every full drain is nevertheless redundant.

Proposition 7.9 (One-pop redundancy). *Let w be an injective push word of length $n > 0$. For valid schedulers, agreement after pushing w and performing $n - 1$ pops is equivalent to agreement on the full n -pop flush. For $n = 1$, no observed pop is needed.*

Proof. Every valid push adds one leaf packet and one matching reference at each internal node on its path. The balance invariant makes all n drain pops succeed, with outputs a permutation of the known input types. After the first $n - 1$ outputs, the last output is the unique input type not yet emitted. Thus the prefix determines the full drain, and equality of prefixes is equivalent to equality of full drains. When $n = 1$, the sole output is already determined by the input. $\square$

Consequently every test in $\mathcal{T}(A)$ may omit its final pop without losing information. No claim is made about the globally sparsest mixture of orientations, pair tests, and still shorter prefixes.

8 An eta-style decision procedure

The decision procedure in Section 7 follows the normalization route: it computes scheduler-independent pair and triple data from which the canonical tree is determined. There is also a direct, pair-dependent route. It recursively compares restrictions of the two input schedulers and constructs only the meet preorders needed for that pair; it never computes either scheduler's normal form. The analogy with eta-equality of functions is deliberately limited but useful: equality is established extensionally, by comparing behavior at the arguments exposed by the two objects, rather than by first normalizing their bodies.

Throughout this section A is a finite named alphabet, transported to $\text{Fin}(k)$ when interfacing with the frozen statement, and all schedulers are valid. We retain the paper's notation F_S for the flush output with **some** erased. By Lemma 3.1, every pop in a flush succeeds, so inequality of these packet words is equivalent to inequality of the exact **List (Option (Fin k))** values returned by **run**.

Four operational facts support the recursion. A run mentioning only $X \subseteq A$ is unchanged when performed in the restriction $S|_X$, so interleaved equivalence is hereditary under restriction. A selector with one active child is online-transparent. Finally, an online-equivalent child may be substituted below a fixed selector: the child receives projected pushes and anonymous pops, and Lemma 3.2 accounts for gaps in its global arrival stamps. These facts preserve exact failed-pop behavior as well as successful outputs.

At every recursive support X with $|X| > 1$, apply Lemma 3.5 to expose

$$\begin{aligned} S|_X &\simeq_{\text{int}} (P, r, (S_B)_{B \in P}), & S_B &\simeq_{\text{int}} S|_B, \\ T|_X &\simeq_{\text{int}} (Q, s, (T_C)_{C \in Q}), & T_C &\simeq_{\text{int}} T|_C, \end{aligned} \tag{50}$$

where P, Q are proper partitions of X . Write $e_S(x, y) = \text{eff}_S(x, y)$ for the effective comparison computed by following the two assigned paths to their first decisive queue. It can be read directly from the finite paths or recovered from the two flushes xy and yx by Table 2; both arrival orders are needed to distinguish a FIFO tie from either strict order.

8.1 The pair-dependent local meet test

Form the incidence meet

$$R = P \wedge Q = \{B \cap C : B \in P, C \in Q, B \cap C \neq \emptyset\}. \tag{51}$$

The local test first compares $e_S(x, y)$ and $e_T(x, y)$ for every pair in distinct R -cells. If they differ, one of xy and yx has different flush output by Table 2; the test runs both and returns the one that differs. Testing every cross-cell pair is essential. On an L-shaped triple one edge can be internal to a P -block and another internal to a Q -block, so testing only pairs separated by both roots would assume the very equivalence being decided.

Suppose all binary tests pass, and write $e(x, y)$ for their common value. Build a directed constraint graph using only pairs in distinct R -cells:

- a strict comparison $x <_e y$ contributes a marked edge $x \rightarrow y$;
- a tie $x \sim_e y$ contributes unmarked edges in both directions.

These constraints extend to a total preorder exactly when no strongly connected component contains a marked edge. In that case, collapse the components, linearly extend the resulting DAG, and assign one natural rank to each component. Members of one component must receive the same rank: merely placing them consecutively would incorrectly turn a required FIFO tie into a strict comparison. The resulting preorder t satisfies

$$\begin{aligned} \text{cmp}_t(x, y) &= \text{cmp}_r(x, y) & \text{if } P(x) \neq P(y), \\ \text{cmp}_t(x, y) &= \text{cmp}_s(x, y) & \text{if } Q(x) \neq Q(y). \end{aligned} \tag{52}$$

Indeed, a P -separated pair is decided at S 's exposed root and a Q -separated pair at T 's; the preceding binary phase makes the two prescriptions agree whenever both roots separate the pair.

If a marked edge lies in an SCC, the cycle itself is only a graph failure, not yet an observational certificate. The shortest-marked-cycle argument behind Lemma 3.7 has the following constructive adaptation. Shorten a marked closed walk whenever vertices two steps apart occupy distinct cells and their three-cell comparisons are transitive. A failed shortcut exposes three distinct cells on which e is not a preorder. If no shortcut and no such triple existed, the walk would alternate between two cells. All comparisons between those fixed cells come from one genuine root preorder— from r if their P -blocks differ, and otherwise from s —so a marked alternating cycle is impossible.

The resulting bad triple must form an L in the $P \times Q$ incidence grid. If all three P -coordinates were distinct, r would govern every edge; if all three Q -coordinates were distinct, s would. Nontransitivity therefore gives distinct a, b, c with

$$a \leq_e b, \quad b \leq_e c, \quad c <_e a. \quad (53)$$

Use the arrival word $\sigma = abc$, whose order orients either of the first two ties forward. Let $\prec_\sigma$ denote the resulting stable pairwise precedence. After possibly exchanging (S, P) with (T, Q) , cyclically name the L's common corner x so that $x \prec_\sigma y \prec_\sigma z \prec_\sigma x$, without changing σ . The possible exchange ensures that y , rather than z , is the neighbor sharing x 's P -block. Thus the P -decomposition groups $\{x, y\} \mid \{z\}$: its root slots are $\{x, y\}, z, \{x, y\}$, its pair child emits x, y , and its output is xzy . The Q -decomposition groups $\{x, z\} \mid \{y\}$: its slots are $\{x, z\}, y, \{x, z\}$, its pair child emits z, x , and its output is zyx . This is the L-diamond calculation of Lemma 3.6, now used after actual binary agreement rather than under an assumed common flush transformer. An implementation need not reconstruct the orientation: it can run all six permutations of the extracted triple and return the first unequal one.

Lemma 8.1 (Local extension or witness). *At a recursive support, the finite local test returns exactly one of:*

1. *a total preorder t satisfying Equation 52; or*
2. *an injective packet word w , with $|w| = 2$ or 3 , such that $F_S(w) \neq F_T(w)$.*

Proof. A binary mismatch gives the second alternative on one of the two arrival orders. After binary agreement, the SCC criterion either constructs t or the marked-walk shortening produces the L-diamond witness above. These are the exhaustive, mutually exclusive branches of the local test. $\square$

8.2 Recursive comparison without normal forms

The procedure $\text{Compare}(X)$ always compares the two original input schedulers restricted to X ; equal subsets are memoized.

1. If $|X| \leq 1$, return an equivalence base certificate.
2. Otherwise expose $S|_X$ and $T|_X$ as in Equation 50, form $R = P \wedge Q$, and run the local test.
3. Return immediately if the local test found a distinguishing word. Otherwise retain its preorder t .
4. Recursively compare every distinct block occurring in $P \cup Q$. A word returned by a child remains a witness at X by restriction exactness.
5. If all children succeed, return a proof node recording P, Q, R, t and references to their certificates.

Both top partitions are proper, so every recursive block is smaller than X . Strong recursion therefore terminates even without memoization; the memo table is needed for the polynomial implementation analyzed below.

Equivalently, for $|X| > 1$ the procedure establishes the exact recursive characterization

$S _X \simeq_{\text{int}} T _X \iff$	$e_S = e_T$ on pairs in distinct $P \wedge Q$ -cells, those comparisons extend to a total preorder t , $S _Y \simeq_{\text{int}} T _Y$ for every distinct $Y \in P \cup Q$.	(54)
--------------------------------------	--	------

The partition, extension, and proof bridge are chosen for this pair at this recursive support. None is a normal form for either input.

8.3 The refinement bridge

The successful local node is sufficient because it supports a one-sided refinement that does not assume the two schedulers equivalent. For every $D \in R$, choose

$$U_D = S|_D, \quad U = (R, t, (U_D)_{D \in R}). \quad (55)$$

For $C \in Q$, let $W_C = U|_C$: after inactive children are removed, this is a $t|_C$ -root over the nonempty cells $D = B \cap C$, with child U_D .

Lemma 8.2 (One-sided refinement). *If the local test returns t , then, for every $C \in Q$,*

$$S|_C \simeq_{\text{int}} W_C. \quad (56)$$

This conclusion does not assume $S \simeq_{\text{int}} T$.

Proof. Restrict the exposed P -root of S to C . Its active colors are exactly the nonempty cells $D = B \cap C$. Below color D , its child is $S_B|_D$. The exposure equivalence in Equation 50, heredity, and restriction idempotence give

$$S_B|_D \simeq_{\text{int}} (S|_B)|_D = S|_D = U_D. \quad (57)$$

This also covers the unary-prefix and terminal-leaf-expansion cases used to obtain the exposure; unary transparency alone would not justify the displayed child equivalence.

Two distinct cells inside the fixed Q -block C lie in distinct P -blocks. Hence Equation 52 says that r and t have the same exact comparison between their colors, including ties. Apply Lemma 3.4 and then substitute the equivalent children from Equation 57. The resulting scheduler is W_C , proving Equation 56. Balance gives common **none** outputs on over-pops. $\square$

Lemma 8.3 (Successful-node sufficiency). *Suppose the local test returns t and recursive certificates prove*

$$E_Y : S|_Y \simeq_{\text{int}} T|_Y \quad \text{for every distinct } Y \in P \cup Q. \quad (58)$$

Then $S|_X \simeq_{\text{int}} T|_X$.

Proof. First transform $T|_X$ into the bridge U . For each $C \in Q$, the exposure, recursive certificate, and one-sided refinement give

$$T_C \simeq_{\text{int}} T|_C \simeq_{\text{int}} S|_C \simeq_{\text{int}} W_C. \quad (59)$$

Substitute W_C below the exposed s -root. This leaves an outer s -selector over Q and, inside each block, a t -selector over the R -cells. By Equation 52 and Lemma 3.4, recolor the outer selector from s to t without changing its Q -trace. The two selector levels now use identical (t, arr) keys and collapse occurrence-for-occurrence as in Section 3.6. Thus

$$T|_X \simeq_{\text{int}} U. \quad (60)$$

For each $B \in P$, restrict Equation 60 to B and use the recursive certificate:

$$S_B \simeq_{\text{int}} S|_B \simeq_{\text{int}} T|_B \simeq_{\text{int}} U|_B. \quad (61)$$

After inactive children are deleted, $U|_B$ is the $t|_B$ -root over the cells $D \subseteq B$. Substitute these restrictions below S 's exposed r -root, recolor r to t , and collapse the duplicated t -levels. This gives $S|_X \simeq_{\text{int}} U$, which combines with Equation 60 to prove the claim.

The collapse couples pending source occurrences, not merely packet labels: the outer queue, the relevant inner queue, and the flat queue choose the same unique (t, arr) -minimum. It therefore covers FIFO ties and repeated packet types. Balance makes selected children nonempty; a common over-pop returns **none** and changes neither state. $\square$

Necessity in Equation 54 is direct. If the two restrictions are interleaved-equivalent, heredity supplies every block equivalence, their two-type restrictions agree, and a marked SCC is impossible because Lemma 8.1 would replay an unequal three-push flush. This argument does not reconstruct either top partition from observations.

The successful-node proof also does not derive refined child transformers from a globally common flush function. Recursive certificates directly supply equivalence on every original P - and Q -block, while Lemma 8.2 supplies the missing bridge. Thus a full cell-autonomy or meet control-word calculation for constructing the refined children is not an implicit premise of this comparator.

8.4 Correctness and small counterexamples

Theorem 8.4 (Eta-style comparison). *For every finite X , $\text{Compare}(X)$ terminates and returns exactly one of:*

- $\text{Equivalent}(c)$, where c proves $S|_X \simeq_{\text{int}} T|_X$; or
- $\text{Distinguishing}(w)$, where w is injective, $|w| \in \{2, 3\}$, and $F_S(w) \neq F_T(w)$.

In particular,

$$\text{Compare}(X) = \text{Equivalent} \iff S|_X \simeq_{\text{int}} T|_X. \quad (62)$$

Proof. Use joint strong induction on $|X|$, proving termination and the stated semantic alternative simultaneously. For $|X| = 0$, every operation is a failed pop. For $|X| = 1$, every successful pop emits the unique type and success depends only on the pending count.

For $|X| > 1$, a binary mismatch is a correct two-letter witness by Table 2; a graph failure is a correct three-letter witness by the L-diamond branch of Lemma 8.1. A word returned from a proper block remains unequal in the original restrictions because it contains no outside pushes, so even its arrival stamps are unchanged. If all recursive calls succeed, Lemma 8.3 constructs the equivalence proof. Every call is on a smaller support, completing the joint induction. Thus no equivalent pair can take a negative branch, since every such branch contains a replayed unequal run, and every positive branch has the bridge proof above. $\square$

Let

$$\mathcal{W}_3(A) = \{w \in A^* : w \text{ is injective and } |w| \in \{2, 3\}\}. \quad (63)$$

Corollary 8.5 (Two- or three-push counterexamples). *For valid stateless schedulers over A ,*

$$S \simeq_{\text{int}} T \iff \forall w \in \mathcal{W}_3(A). F_S(w) = F_T(w). \quad (64)$$

Consequently, if $S \not\simeq_{\text{int}} T$, there is a differing flush word with exactly two or three pushes of pairwise-distinct packet types.

Proof. The forward implication is immediate. For the reverse implication, apply the induction in Theorem 8.4. Binary equality supplies the common cross-meet comparisons; if they did not extend, the L-diamond would give a word in $\mathcal{W}_3(A)$. Every proper P - or Q -block inherits the same hypothesis, so its recursive comparison succeeds, and Lemma 8.3 finishes. Equivalently, run the comparator: its only negative leaves are words in $\mathcal{W}_3(A)$. $\square$

A simple streaming implementation may therefore test

$$2\binom{k}{2} + 6\binom{k}{3} = k(k-1)^2 \quad (65)$$

words and stop on the first difference. This observational implementation does not materialize a profile, but its correctness proof is the recursive meet argument above. The structural comparator can do less work on shallow inputs.

8.5 Memoized complexity

A machine bound requires a representation for the mathematical assignment function. Assume the topology and all assigned paths are finite encoded data; let N be their read and tabulation cost, measured in words for a word-RAM bound and in bits for a bit-cost bound. Validity is promised or checked during preprocessing. Type and cluster identifiers and compressed rank classes fit one word, and b is the maximum bit length of an input rank when bit cost is charged. Thus inactive topology and long common unary paths are included in N , rather than treated as free. The asymptotic discussion below assumes $k \geq 2$; the cases $k = 0, 1$ terminate at the base case after the input is read.

For each input scheduler, contract active unary nodes and apply the terminal leaf expansion from Lemma 3.5 recursively. The result is an *active support hierarchy*: a rooted laminar tree with the k types as singleton leaves and no unary internal node. It has at most $k - 1$ internal nodes for $k \geq 1$. This is syntax-directed indexing of the input, not a semantic canonical form. At every active selector, retain the rank class of each descendant type. For input $i \in \{1, 2\}$, put

$$a_i(x) = \#\{\text{active support clusters on } x\text{'s ancestor chain}\}, \quad L_i = \sum_x a_i(x). \quad (66)$$

Up to the harmless convention for singleton clusters,

$$k \leq L_i \leq k^2. \quad (67)$$

Rank compression at the selectors costs $O(bL_i \log k)$ bit operations by comparison sorting. LCA and selector/type lookup tables then make an effective two-type comparison constant-time.

Every subset reached by the recursion has the form

$$X' = X_1 \cap X_2, \quad (68)$$

where X_i is a cluster in input i 's active hierarchy. Indeed, the exposed top of $S|_{X'}$ is the least first-hierarchy cluster containing X' , and its blocks are intersections of X' with that cluster's children; the second hierarchy is symmetric. Starting from the full alphabet, recursion on a P - or Q -block therefore preserves the intersection form.

Replacing X_1, X_2 by the least clusters containing X' gives a canonical memo key and preserves the intersection. If $X' = X_1^0 \cap X_2^0$, the two least clusters satisfy $X_i \subseteq X_i^0$ while still containing X' , so their intersection is exactly X' . Each hierarchy has $O(k)$ clusters, so there are only $O(k^2)$ nonempty memo states. Memoization is crucial: opposite combs can expose two different $(m - 1)$ -blocks and make the unfolded recurrence satisfy

$$T(m) \geq 2T(m - 1) + O(m), \quad (69)$$

so the literal recursive call tree can be exponential even for equivalent inputs.

Scanning every pair independently in every state gives the safe but loose bound $O(N + k^4)$. Instead enumerate exactly the pairs separated by the current P or Q , which are the pairs in distinct meet cells. In the sums below, $x < y$ uses the fixed alphabet enumeration only to select each unordered pair once. Fix types x, y . At a state where P separates them, the first hierarchy's cluster coordinate is forced to $\text{lca}_1(x, y)$; the other coordinate can be only a common ancestor of x, y , giving at most $\min(a_2(x), a_2(y))$ choices. The symmetric charge applies when Q separates them. Hence the total number of generated pair constraints is

$$\begin{aligned} E_{\text{all}} &= O\left(\sum_{x < y} \min(a_1(x), a_1(y)) + \sum_{x < y} \min(a_2(x), a_2(y))\right) \\ &= O(k(L_1 + L_2)). \end{aligned} \quad (70)$$

The last step uses $\min(u, v) \leq (u + v)/2$; in the unordered-pair sum each $a_i(x)$ occurs $k - 1$ times. The total number of type occurrences over all memo states also satisfies

$$Z \leq \sum_x a_1(x)a_2(x) \leq k \min(L_1, L_2). \quad (71)$$

Partition construction, pair checking, SCCs, and topological extension are linear in these state vertices and edges. Shortening a marked cycle to its bad triple costs $O(|X'|^2)$ within the single failing state and remains within the same aggregate bound. A simpler implementation may enumerate triples instead; this adds $O(k^3)$, preserves the cubic worst-case bound, but can lose the sharper quadratic bound on a shallow failing input.

Theorem 8.6 (Eta comparator complexity). *After input and rank preprocessing, the optimized memoized comparator uses*

$$O(k(L_1 + L_2)) \subseteq O(k^3) \quad (72)$$

word-RAM operations. Its working decision space, including the encoded input but excluding a retained positive certificate, can be $O(N + k^2)$ by storing the memo keys and one local graph at a time.

The word bound assumes that an identifier, graph endpoint, memo key, and compressed rank class occupy $O(\log k)$ bits. Charging their bit operations and the comparison sorting gives the conservative combined bound

$$O(N + b(L_1 + L_2) \log k + k(L_1 + L_2) \log k), \quad (73)$$

where N is measured in bits. In particular, the structural word term in Equation 72 acquires a $\log k$ factor; unbounded natural ranks are not silently unit-cost. When both hierarchies are shallow, $L_1 + L_2 = \Theta(k)$, the direct structural work is $O(k^2)$. Opposite combs can make $L_i = \Theta(k^2)$ and attain the cubic charge.

This adaptive improvement is not a subcubic worst-case claim. For comparison, one can store one three-valued code for every unordered pair and six flush outputs, one for each input permutation of every unordered triple, for

$$\binom{k}{2} + 6 \binom{k}{3} = \Theta(k^3) \quad (74)$$

constant-size codes. After the same hierarchy and LCA preprocessing, a two- or three-type restriction has only constantly many active branching selectors, reached by jumping to the LCA of the selected types and, when needed, of the remaining pair. Its drain therefore uses $O(1)$ compressed-rank comparisons. The scheduler-independent profile takes $O(k^3)$ additional time to write and is reusable in later comparisons. The direct comparator has the same cubic worst-case exponent, but avoids materializing two dense profiles, may stop at the first witness, and can take only quadratic structural work on shallow pairs. The $\Omega(k^3)$ output cost applies only when that explicit dense profile is written, and the opposite-comb charge analyzes this implementation; neither is a lower bound for every exact algorithm or canonical syntax. In particular, no claim is made that worst-case $o(k^3)$ decision or representation after input reading and rank compression is impossible.

8.6 Certificates in both directions

On inequivalence, the returned w is injective and has two or three letters. The certificate is the operation word $\text{flushOps}(w)$, of length four or six. A checker simply runs both original schedulers and verifies that their exact `List (Option (Fin k))` outputs differ. Once replayed, this certificate is independent of the comparator's correctness proof. Its constant operation length follows because primitive failures are binary-table failures or L-diamonds and restriction lifting never changes the word. Its two or three type identifiers still occupy $O(\log k)$ bits each.

There is no single positive test word. An equivalence certificate is a finite, pair-dependent proof DAG keyed by the reachable cluster intersections in Equation 68. Every non-base node records

- its partitions P, Q, R ;
- rank classes for t , preserving every required cross-cell strict comparison and tie; and
- references to certificate nodes for every distinct block in $P \cup Q$.

The checker recomputes both exposures, checks the recorded $R = P \wedge Q$, and recomputes both effective comparisons for every required cross-cell pair. It checks equality, verifies t against the common strict-or-tie value, checks that every required proper block is referenced, and applies the fixed one-sided-refinement,

substitution, recoloring, and collapse inference from Section 8.3. Supplying t avoids constructing an extension but not checking its constraints. It scans the same Z occurrences and E_{all} pair constraints as successful decision, so its checking cost after preprocessing is

$$O(Z + E_{\text{all}}) = O(k(L_1 + L_2)) \quad (75)$$

word operations, cubic for opposite combs. Storing every chosen t and the DAG metadata takes $O(Z \log k)$ bits, at most $O(k^3 \log k)$.

The two certificate directions have different modes of checking. The negative word is independently replayable against the frozen run definition. Development 16 of the pinned verification manifest encodes the comparator as a terminating computable definition and kernel-checks its soundness, completeness, and two-or-three-push witness-size theorem; its soundness and witness-size gates use only `{proptext, Quot.sound}`. Its trust-level-zero audit passes, and its `#eval` probes execute the comparator and replay separating witnesses through the frozen semantics. In the positive direction, `etaCompare_complete` validates the comparator's equivalent outcome. A positive proof DAG cannot be validated merely by replaying scheduler outputs; its recursive inference is instead covered by that kernel-checked completeness theorem. A common dense profile of Equation 74, together with replay of all its entries and Corollary 8.5, is an alternative canonical positive certificate. The proof DAG, like the comparator itself, remains pair-dependent. This is the precise eta-style advantage: it decides equivalence without first computing normal forms, while making no claim of a better worst-case exponent.

9 Confluence and bottom-up normalization

The canonical tree of Section 6 is a semantic normal form, but it is not the irreducible form of the natural equations used in the proofs above. This distinction is sharp. The bidirectional rewrite inventory is locally confluent for a generic equational reason, yet it is nonterminating and even a canonical tree has an outgoing step. In contrast, a compositional bottom-up evaluator does compute the canonical form, provided that each fold value carries the complete restriction-ready pair-and-triple summary.

9.1 The active rewrite inventory

Fix a finite nonempty support X . In the raw restriction, annotate every subtree by the types whose paths visit it; inactive pruning `I` acts on this raw supported syntax. After inactive child slots have been deleted and indices remapped consistently, the remaining rules use the active syntax

$$K ::= L_\lambda[X] \mid N_p(P; (K_B)_{B \in P}), \quad (76)$$

where P partitions X into nonempty child supports, $p : X \rightarrow \mathbb{N}$ is the selector rank map, and $\lambda : X \rightarrow \mathbb{N}$ is a leaf rank map. Equivalently, `U` may match a raw supported node with exactly one nonempty-support child, contracting it while discarding its inactive siblings. This raw-syntax form makes the `I/U` overlap below well typed. Zero support is handled separately by the fixed $E_0 = \text{node}[]$ from Equation 33.

The relation $\rightarrow_{\mathcal{R}}$ is the contextual closure on the supported raw/active syntax of the following inventory. Equality in every side condition is equality of the full three-way rank comparison; FIFO is still determined by arrival stamps.

1. *Inactive pruning* (`I`) deletes an empty-support child, remaps larger indices, and replaces a wholly zero-support term by E_0 .

2. *Unary contraction* (`U`) is directed:

$$N_p(\{X\}; K_X) \longrightarrow K_X. \quad (77)$$

3. *Rank swap* (`RS`) is bidirectional:

$$N_p(P; (K_B)) \longleftrightarrow N_q(P; (K_B)), \quad (78)$$

when $\text{cmp}_p(x, y) = \text{cmp}_q(x, y)$ for types in distinct P -blocks. This is the colored-PIFO law Lemma 3.4.

4. *Context (CTX)* lifts one local child step through an unchanged parent. Arbitrary semantic child equivalence is not itself an atomic rewrite step.
5. *Equal-key collapse/regrouping (EC/RG)* is bidirectional. If R refines P , then

$$\mathbf{N}_t(P; (\mathbf{N}_t(R|_B; (K_D)_{D \subseteq B}))_{B \in P}) \longleftrightarrow \mathbf{N}_t(R; (K_D)_{D \in R}). \quad (79)$$

The same type rank t occurs at both levels, so the selectors remove the same unique (t, arr) -minimum occurrence.

6. *Leaf explosion/contraction (LE/LX)* is bidirectional. For any finite partition Q of X ,

$$\mathbf{L}_\lambda[X] \longleftrightarrow \mathbf{N}_s(Q; (\mathbf{L}_{\lambda|C}[C])_{C \in Q}), \quad (80)$$

provided s and λ have the same comparisons across distinct Q -blocks. The inventory permits the one-block partition.

A behavioral decomposition can also be *realized* at the root, but this promotion is a derived, recursively constructed finite conversion using the rules above. Its side condition quantifies over the flush transformer, so it is not another primitive local rule.

Definition 9.1 (Representation equivalence). Let $\equiv_{\text{rep}}$ be the least contextual congruence generated by permuting siblings with the corresponding child-index remapping, replacing an internal rank map while preserving every cross-block comparison, and replacing a leaf rank map while preserving its weak order. This equivalence preserves the active block-labelled shape; in particular, it cannot insert or delete selector levels.

The larger relation $\mathcal{N}(S) = \mathcal{N}(T)$ is deliberately not part of $\equiv_{\text{rep}}$: by Theorem 6.7 it is already semantic interleaved equivalence, so using it as the rewrite quotient would assume the desired normalization result.

For the termination question, use the explicit existential class step

$$[K]_{\text{rep}} \Rightarrow_{\mathcal{R}} [L]_{\text{rep}} \iff [K]_{\text{rep}} \neq [L]_{\text{rep}} \wedge \exists K' \equiv_{\text{rep}} K, L' \equiv_{\text{rep}} L. K' \longrightarrow_{\mathcal{R}} L'. \quad (81)$$

This is a relation on classes; no representative-coherent implementation is being assumed.

9.2 The rewrite relation does not terminate

Theorem 9.2 (Failure of rewrite normalization). *The literal relation $\longrightarrow_{\mathcal{R}}$ and the existential quotient relation $\Rightarrow_{\mathcal{R}}$ are nonterminating. No well-founded measure can strictly decrease on every supplied rewrite step, and the semantic canonical forms are not the irreducible terms of this relation.*

Proof. Let

$$A = \{a, b, c, d\}, \quad R = \{\{a\}, \{b\}, \{c\}, \{d\}\}, \quad P = \{\{a, b\}, \{c, d\}\},$$

let $t(x) = 0$ for every type, and let every $K_{\{x\}}$ be a singleton leaf. Put

$$\begin{aligned} T &= \mathbf{N}_t(R; (K_D)_{D \in R}), \\ U &= \mathbf{N}_t(P; (\mathbf{N}_t(R|_B; (K_D)_{D \subseteq B}))_{B \in P}). \end{aligned}$$

Regrouping and collapse give

$$T \xrightarrow{\text{RG}} U \xrightarrow{\text{EC}} T. \quad (82)$$

The two classes remain distinct modulo $\equiv_{\text{rep}}$: T has one selector level, whereas U has an outer selector and two nontrivial inner selectors. A strictly decreasing measure would therefore imply $\mu([T]) > \mu([U]) > \mu([T])$.

There is also a two-step instance of the unrestricted leaf rule. For any nonempty X , choose $Q = \{X\}$ in Equation 80; its side condition is vacuous, and

$$\mathbf{L}_\lambda[X] \xrightarrow{\text{LE}} \mathbf{N}_s(\{X\}; \mathbf{L}_\lambda[X]) \xrightarrow{\text{U}} \mathbf{L}_\lambda[X]. \quad (83)$$

Forbidding this one-block explosion would not remove Equation 82.

Finally, T is already canonical up to representation. Every type has a distinct singleton root child, so its rooted-obstruction graph is edgeless, its component partition is exactly R , and its children are singleton leaves. Thus $T \equiv_{\text{rep}} N(T)$, yet the RG step in Equation 82 leaves it. Hence “canonical” does not mean “irreducible.” $\square$

The topology potential used by a recursive realization construction is not a rewrite measure. Rank swap preserves topology, leaf explosion and regrouping add selector levels, and collapse removes them. That potential proves that one selected conversion is finite, not that every contextual rewrite chain terminates. Newman’s lemma therefore does not apply, and the proposed identity

$$\{\text{irreducible terms of } \mathcal{R}\} = \{N(S)\} / \equiv_{\text{rep}}$$

is false. This refutes the supplied unrestricted inventory, not the possible existence of a different phased or target-directed calculus.

9.3 What the critical pairs do show

The overlap ledger is useful but should not be mistaken for a terminating critical-pair proof. Representative cases are as follows.

- Two inactive deletions commute after index adjustment; pruning against unary contraction reaches the same active child; two nested unary contractions reach the same grandchild.
- Two rank swaps at one node swap directly between the two target rank maps. A rank-swap/collapse peak first returns to the common t representative and then collapses.
- Two leaf explosions through partitions P, Q join at

$$N_\lambda(P \wedge Q; (L_{\lambda|D})_{D \in P \wedge Q}). \quad (84)$$

First rank-swap each branch’s outer selector to λ , then refine its children to the meet and collapse the duplicate λ -levels.

- Two adjacent collapses flatten to the finest displayed partition. Two regroupings can instead collapse back to their common source. An EC/RG peak at one boundary consists of literal inverse steps.
- A descendant step commutes with collapse or regrouping by CTX; the same child occurs on both sides.

These examples are representative, not an exhaustive positional table. The generic argument supplies local confluence. If both arms of a one-step peak are reversible, either endpoint returns to the source and follows the other arm. If exactly one arm is I or U , reverse the other arm and then eliminate. If both arms are eliminations, disjoint redexes commute and nested redexes reduce to the deletion/splicing diamonds above. Thus the literal relation is locally confluent for an equational reason. The same argument proves local confluence of the existential relation on $\equiv_{\text{rep}}$ -classes. It does not prove the stronger coherence needed to execute collapse independently of the chosen representative, and several joins run backward. It therefore supplies no orientation or useful rewrite-normal-form theorem.

9.4 A compositional semantic normalizer

The positive result uses a different interface. A fold value on support X carries

$$D_X = (E_X, H_X), \quad E_X(x, y) = \text{eff}(x, y), \quad H_X(x, y \mid z) = R(x, y \mid z). \quad (85)$$

These tables are restriction-ready: the summary on $Y \subseteq X$ is simply $D_X|_Y$. Pair data alone does not suffice, as Example 7.5 shows. Write

$$\text{bad}(a, b, c) \iff \text{comp}(a, b) = \text{some}(d) \text{ for some } d \neq c.$$

At a leaf $\mathsf{L}_\lambda[X]$, the summary algebra is

$$E(x, y) = \text{cmp}_\lambda(x, y), \quad H(x, y \mid z) = \text{false}. \quad (86)$$

At a node $M = \mathsf{N}_\rho(P; (M_B)_{B \in P})$, let $b(x)$ be the active child block of x , and suppose the children provide (E_B, H_B) . Then

$$E(x, y) = \begin{cases} E_B(x, y), & b(x) = b(y) = B, \\ \text{cmp}_\rho(x, y), & b(x) \neq b(y), \end{cases} \quad (87)$$

and, for pairwise-distinct x, y, z ,

$$H(x, y \mid z) = \begin{cases} H_B(x, y \mid z), & b(x) = b(y) = b(z) = B, \\ \text{bad}(E(x, z), E(z, y), E(x, y)), & b(x) = b(y) \neq b(z), \\ \text{false}, & b(x) \neq b(y). \end{cases} \quad (88)$$

The middle line uses the child's effective two-type comparison and therefore already accounts for reference hijacking; it does not identify a parent reference with the packet eventually emitted below it.

For a certified realizable summary $D = (E, H)$, define $\text{Canon}(X, D)$ by the reconstruction of Definition 6.4. It returns E_0 when X is empty and a rank-zero leaf when X is a singleton. Otherwise form the graph

$$x - y \iff \exists z \in X \setminus \{x, y\}. H(x, y \mid z) \vee H(y, x \mid z), \quad (89)$$

recurse on its connected components C with summary $D|_C$, and choose root ranks q satisfying

$$C \neq C', \ x \in C, \ y \in C' \implies \text{cmp}_q(x, y) = E(x, y). \quad (90)$$

Cell feasibility Lemma 6.3 supplies these ranks; fixed sibling ordering and a fixed topological ordering of the strict rank classes select one deterministic representative.

The smart constructors are now simple.

- $\text{mkLeaf}(X, \lambda)$ computes Equation 86 and returns $\text{Canon}(X, D)$. For $|X| > 1$ this is a discrete singleton-child selector ordered by λ ; for $|X| = 1$ it resets the irrelevant leaf rank to zero; for $X = \emptyset$ it returns E_0 .
- mkNode discards inactive children. With zero active blocks it returns E_0 ; with one block it returns that normalized child; with at least two blocks it computes Equation 87 and Equation 88, then returns $\text{Canon}(X, D)$.

In particular, a normal value includes the exact summary of every restriction. Its concrete restriction need not itself remain structurally normal.

Precise bottom-up normal invariant. A fold value (T, E, H) on support X satisfies $\text{NF}(X)$ when all of the following hold.

1. T is a valid concrete scheduler with active support exactly X .
2. T is active-reduced: it has no inactive children and no active unary nodes.
3. $T \equiv_{\text{rep}} \text{Canon}(X, (E, H))$.
4. $E = \text{eff}_T$ and $H = R_T$.
5. For every $Y \subseteq X$, $(E|_Y, H|_Y)$ is the exact summary of $T|_Y$; the concrete restriction $T|_Y$ need not itself remain structurally normal.
6. Structurally, the empty case is E_0 , singleton cells are leaves, and every larger cell has as its child supports the pairwise-disjoint components of Equation 89, covering its support. Its cross-child ranks realize E , and each child on C has summary $(E|_C, H|_C)$ and recursively satisfies this invariant.

Theorem 9.3 (Bottom-up canonical normalization). *Prune inactive syntax and fold `mkLeaf` and `mkNode` over any valid finite-alphabet scheduler S on A . The resulting fold value $(\text{norm}(S), \text{eff}_S, R_S)$ satisfies $\text{NF}(A)$. Its abstract tree is exactly $\mathcal{N}(S)$, its concrete tree is $N(S)$ up to $\equiv_{\text{rep}}$, and*

$$S \simeq_{\text{int}} \text{norm}(S).$$

Proof. Induct on the active input topology. The leaf equations are exact, and `mkLeaf` reconstructs their canonical tree. Rank zero is sound on a singleton because its queue is ordered only by FIFO arrival.

At a node, first apply the induction hypothesis to every active child. Child congruence preserves the parent's run: the unchanged parent sends corresponding children the same projected pushes and anonymous pops. If no block remains, the support is empty and the result is E_0 . If one block remains, both node equations reduce pointwise to the child's tables, unary transparency removes the parent, and the child induction hypothesis applies unchanged. If at least two blocks remain, Equation 87 and Equation 88 compute the exact pair/triple data of the composite. Canonical normalization Theorem 6.5 makes reconstruction from those data interleaved-equivalent, while Theorem 6.7 identifies the resulting abstract tree with $\mathcal{N}(S)$. Restriction locality of the two tables preserves the fold invariant for the parent. $\square$

9.5 Why a smart node may have to reopen a normal child

The call to `Canon` in `mkNode` cannot in general be implemented by wrapping immutable normal child roots. Consider the already-normal child on $B = \{x, y, z, u\}$ with top blocks

$$\{x, y\} \mid \{z\} \mid \{u\}, \quad \rho(u) = 0, \rho(x) = 1, \rho(z) = 2, \rho(y) = 3, \quad (91)$$

and an inner singleton selector on $\{x, y\}$ ordered $y < x$. Its data include

$$E(x, y) = >, \quad E(x, z) = <, \quad E(z, y) = < ,$$

so $H(x, y \mid z)$ holds, and its canonical top components are exactly the three displayed blocks.

Place this child below a new parent beside a singleton w , with

$$p(x) = 0, \quad p(w) = 1, \quad p(y) = p(z) = p(u) = 2. \quad (92)$$

The old $x - y$ obstruction remains, and

$$E_B(x, u) = >, \quad E(x, w) = <, \quad E(w, u) = <$$

makes w witness $H(x, u \mid w)$. The five-type component partition is therefore

$$C = \{x, y, u\}, \quad \{z\}, \quad \{w\}. \quad (93)$$

This list is exhaustive. Through witness w , the $x - z$ comparisons compose to the already equal value and create no edge, while every pair among y, z, u has opposite strict signs and hence no composition; any rooted pair containing w is separated at the parent. The old child contributes only the $x - y$ edge.

When reconstructing the child on C , restriction removes both witnesses z and w . The only remaining candidate inside an old shared block is x, y , but

$$E(x, u) = >, \quad E(u, y) = <$$

has opposite strict signs; pairs involving u were separated by the old child root. Its obstruction graph is therefore edgeless, so $N(C)$ is a one-level singleton selector ordered $u < y < x$. Merely restricting the old normal child instead leaves a two-level tree with outer blocks $\{x, y\} \mid \{u\}$ and an inner $x \mid y$ selector. The parent constructor must therefore use the cached restriction summary to dissolve and reblock structure inside that child.

The conclusion is semantic rather than literal rewrite reachability to the chosen deterministic representative. Decomposition realization yields at most a finite local-equation conversion from S to some C_{real} with

$$C_{\text{real}} \equiv_{\text{rep}} \text{norm}(S). \quad (94)$$

For example, $L_7[\{x\}]$ normalizes to the chosen $L_0[\{x\}]$, but none of the primitive rules changes a terminal leaf rank: leaf explosion copies it, rank swap changes an internal selector, collapse/regrouping rearranges selectors, and pruning/contraction only remove structure. Thus Theorem 9.3 asserts semantic equivalence and equality of the abstract canonical object, not a literal rewrite path to **Canon**.

9.6 Cost

Use a unit-cost RAM model for child indices and natural-rank comparisons, with every assigned path represented explicitly. Let $k = |A|$, let n and d_{raw} be the raw topology size and depth, let h be the maximum active path length (zero when $k = 0$), and let k_v be the number of active types visiting node v . Here n includes inactive subtrees visited by the full validation pass. That validation and pruning pass costs $O(n + kh)$ time and an $O(d_{\text{raw}})$ stack. The pair/triple algebra at v costs $O(k_v^3)$, and charging a type triple to its common active ancestors gives

$$\sum_v k_v^3 \leq k^3(h + 1). \quad (95)$$

With persistent summaries, restriction views, and one final materialization, the total time is

$$O(n + k^3(h + 1)). \quad (96)$$

The concrete output has $O(k + 1)$ topology nodes and $O(k(h + 1))$ path/rank entries. Eagerly rebuilding a complete concrete normal tree at every intermediate constructor has the safer bound

$$O(n + k^3(h + 1)^2). \quad (97)$$

Postorder release of child tables and zero-copy restriction views give $O(1 + k^3 + kh + d_{\text{raw}})$ auxiliary space; copying every restricted triple table instead gives $O(1 + k^3(h + 1) + kh + d_{\text{raw}})$. For trusted pointer-resident input, path-guided pruning omits the n and d_{raw} terms. Bit-cost accounting additionally charges the encoded-rank comparison cost.

Thus rewriting cannot orient the theory as supplied: its useful equations are reversible and cyclic. Semantic bottom-up normalization succeeds because it evaluates the complete local summary and reconstructs the canonical object, rather than searching for an irreducible term.

10 Normalization by evaluation

We now give a second proof of Theorem 2.2, organized as normalization by evaluation (NbE). The semantic value of a scheduler is its literal flush transformer; no topology is retained. Semantic factor partitions then support a canonical reification, and a common-refinement argument proves that reification preserves every interleaved observation.

Throughout this section A is finite and carries a fixed enumeration; in the frozen statement, $A = \text{Fin}(k)$. We use the notation from Section 3: $F_S : A^* \rightarrow A^*$ is the drain of a valid scheduler S , $w|_B$ filters a word to $B \subseteq A$, $\pi_P(w)$ is its word of blocks for a partition P , and $\text{sort}_a(w)$ stably sorts occurrences by type rank $a : A \rightarrow \mathbb{N}$ and source arrival. By Lemma 3.1,

$$\text{run}(S, \text{flushOps}(w)) = \text{map}(\text{some}, F_S(w)).$$

Thus $F_S(w)$ is a content-preserving permutation of the occurrences of w . Occurrence identities need not be stored separately: equal-type occurrences have identical paths and ranks and emerge in arrival order, so the j -th output occurrence of a type is its j -th input occurrence.

10.1 Drain semantics and factors

Definition 10.1 (Semantic domain and evaluation). Define the realizable semantic domain by

$$\text{Sem}(A) = \{F : A^* \rightarrow A^* : F = F_S \text{ for some valid stateless scheduler } S\}. \quad (98)$$

Evaluation is literal draining:

$$\llbracket S \rrbracket = F_S. \quad (99)$$

Consequently, for valid schedulers,

$$\llbracket S \rrbracket = \llbracket T \rrbracket \iff S \simeq_{\text{flush}} T.$$

The reification phase must recover enough hierarchy from an extensional $F \in \text{Sem}(A)$. Its basic units are the following semantic partitions. For a partition P , write $P(x)$ for the unique block containing x .

Definition 10.2 (Factor partition). A partition P of A is a *factor* of $F \in \text{Sem}(A)$ when both of the following hold.

1. *Block locality*: for every $B \in P$ and $w \in A^*$,

$$F(w)|_B = F(w|_B). \quad (100)$$

2. *Stable control*: some $a : A \rightarrow \mathbb{N}$ satisfies, for every w ,

$$\pi_P(F(w)) = \pi_P(\text{sort}_a(w)). \quad (101)$$

We call a a control witness. It need not be constant within a block.

The indiscrete partition is always a factor; on the empty alphabet it is the unique empty partition. At an internal root, the partition into used children is a factor. Indeed, the root ranks give the control witness, and its stable drain fixes the word of child invocations. Because every push precedes every pop, the subsequence emitted from a child is exactly the full drain of the input subword routed there, which gives locality. The child's global arrival stamps may have gaps, but their order is unchanged. A leaf will be treated as its online-equivalent presentation with a ranked root and singleton children, as in Lemma 3.5.

10.2 The factor meet

The central semantic fact is stronger than merely amalgamating two particular root decompositions: factors are closed under common refinement.

Lemma 10.3 (Factor meet). *Let $F \in \text{Sem}(A)$, and let P, Q be factors of F . Then*

$$R = P \wedge Q = \{B \cap C : B \in P, C \in Q, B \cap C \neq \emptyset\} \quad (102)$$

is also a factor of F .

Proof. Locality. For $D = B \cap C \in R$, filters commute, and the two factor-locality equations give

$$\begin{aligned} F(w)|_D &= (F(w)|_B)|_C = F(w|_B)|_C \\ &= F((w|_B)|_C) = F(w|_D). \end{aligned} \quad (103)$$

The two controls. Let a witness P and b witness Q . Identify each nonempty R -cell with its pair of containing blocks. Position by position,

$$\pi_R(F(w)) = \text{zip}(\pi_P(\text{sort}_a(w)), \pi_Q(\text{sort}_b(w))). \quad (104)$$

The zip is not an arbitrary pairing: its two coordinates are the P - and Q -projections, at the same positions, of the single permutation $F(w)$. It therefore has exactly the multiset of nonempty R -cells occurring in w , for every word and every choice of multiplicities.

For types in distinct R -cells, impose the exact comparison of $a(x)$ and $a(y)$ when P separates x, y , and the exact comparison of $b(x)$ and $b(y)$ when Q separates them. Exactness includes $<, =, >$. If both partitions separate the pair, the two requirements agree. Otherwise one of the two arrival orders xy, yx in Equation 104 would pair the diagonal coordinates into an absent off-diagonal cell, including when one control ties the pair and the other orders it strictly.

No finite obstruction. Represent a strict requirement $x < y$ by a flagged edge $x \rightarrow y$, and a required tie by unflagged edges in both directions. These constraints extend to a weak order precisely when there is no closed directed walk containing a flagged edge. Suppose there is one and choose a shortest such walk. It is a simple cycle apart from its repeated endpoint. A two-cycle is the pairwise conflict just excluded.

For a longer shortest cycle, two nonconsecutive vertices in distinct R -cells have a comparison chord. A strict chord and the oppositely directed cycle arc give a shorter flagged cycle; for a tie chord, choose the arc containing the original flagged edge. Thus every two nonconsecutive vertices of a shortest obstruction must occupy one R -cell. A cycle of length at least five is then impossible: x_0, x_2, x_3 would occupy one cell, contradicting the comparison edge $x_2 \rightarrow x_3$. In a four-cycle, the opposite vertices form two cells. Either P separates these cells, in which case all four edges come from a , or Q does, in which case they all come from b . Neither weak order contains a flagged cycle.

It remains to exclude a triangle. If either partition separates all three pairs, one of a, b governs the triangle. The only remaining incidence shape, up to relabeling, is the L

$$P(x) = P(y) \neq P(z), \quad Q(y) = Q(z) \neq Q(x). \quad (105)$$

Put

$$(u, v, t) = (\text{cmp}_b(x, y), \text{cmp}_a(y, z), \text{cmp}(z, x)), \quad (106)$$

where the last comparison is common to a and b . Of the 27 triples, 13 are compatible with a weak order. For each of the other 14, Table 4 gives an arrival word for which the zip in Equation 104 contains the missing fourth $P \times Q$ cell, or has the wrong cell multiplicity. This contradicts the all-arity content condition following Equation 104.

(u, v, t)	arrival word	(u, v, t)	arrival word
$(<, <, <)$	xyz	$(>, >, >)$	xyz
$(<, <, =)$	yzx	$(>, >, =)$	xyz
$(<, =, <)$	xyz	$(>, =, >)$	xzy
$(<, =, =)$	yzx	$(>, =, =)$	xzy
$(=, <, <)$	xyz	$(=, >, >)$	yxz
$(=, <, =)$	zxy	$(=, >, =)$	yxz
$(=, =, <)$	xyz	$(=, =, >)$	zyx

Table 4: The fourteen inconsistent L-shaped controls. Changing the arrival word is essential in the rows involving FIFO ties.

There is no flagged cycle. Collapse the tie strongly connected components and topologically order the resulting strict DAG; natural ranks for this order give a weak priority g satisfying every imposed comparison. Exact cross-block comparisons determine a colored stable sort, so

$$\pi_P(\text{sort}_g(w)) = \pi_P(\text{sort}_a(w)), \quad \pi_Q(\text{sort}_g(w)) = \pi_Q(\text{sort}_b(w)). \quad (107)$$

Zippping these equalities and using Equation 104 yields

$$\pi_R(F(w)) = \pi_R(\text{sort}_g(w)),$$

which is stable control for R . Together with Equation 103, this proves that R is a factor. $\square$

The all-word condition in this proof cannot be replaced by the collection of two-type restrictions.

Example 10.4 (Pairwise flush data is insufficient). On types A, B, C , consider the two two-leaf schedulers

	root routing and ranks	ranks in nonsingleton child
S_1	$\{A, B\} \mid \{C\}, \quad A = 1, B = C = 2$	$A = B$
S_2	$\{A, C\} \mid \{B\}, \quad A = B = C$	$A < C$.

(108)

All leaf ranks of S_1 tie; the singleton leaf ranks are immaterial. On every word over each displayed pair, repetitions included, their drains agree:

$$\{A, B\} : \text{FIFO}, \quad \{A, C\} : A \text{ stably before } C, \quad \{B, C\} : \text{FIFO}. \quad (109)$$

Nevertheless the ternary word distinguishes them:

$$F_{S_1}(CBA) = BCA, \quad F_{S_2}(CBA) = ABC. \quad (110)$$

Thus complete pairwise flush behavior does not determine a flush transformer; the arbitrary-multiplicity content constraint in the factor-meet proof is essential.

10.3 Reification and adequacy

There are finitely many partitions of finite A . The indiscrete partition is a factor, and repeated application of Lemma 10.3 shows that the meet of all factors is a factor. Call this finest factor P_F . When $A \neq \emptyset$, among the control witnesses whose ranks have been replaced by consecutive rank classes, choose the lexicographically least compressed weak-order witness

$$g_F : A \rightarrow \{0, \dots, m-1\}.$$

Here m is its number of nonempty rank classes. This choice is from a finite set and depends only on F . On the empty alphabet take the unique empty function.

If $|A| > 1$, P_F is proper. To see this, take any scheduler realizing F , discard unused children, and contract roots with only one used child. A branching root supplies a proper root factor. If the first live node is a leaf, then $F = \text{sort}_a$ for its leaf ranks a , and the discrete singleton partition is a proper factor. Since P_F refines every factor, it is proper as well.

We may therefore define $\text{reify}_A(F)$ by recursion on $|A|$. For $|A| \leq 1$, use a rank-zero leaf, with the vacuous assignment when A is empty. Otherwise, make a root with one canonically ordered child for every $B \in P_F$, and give type x root rank $g_F(x)$. The child for B is

$$\text{reify}_B(F|_B), \quad F|_B : B^* \rightarrow B^*, \quad F|_B(v) = F(v).$$

Factor locality and content preservation make the displayed restriction well typed. It is realizable by restricting any witness for F to types in B , and the recursive call is smaller because P_F is proper. The root annotation followed by the recursively produced path is well formed, so the result is a valid scheduler.

Lemma 10.5 (Increasing arrival renaming). *In a queue or tree computation, replace the global arrival stamps of source pushes by any strictly increasing relabeling, using the same new stamp at every node visited by one push. Every emitted type is unchanged.*

Proof. Ranks are unchanged, and a strictly increasing relabeling preserves every lexicographic (rank, arr) comparison. Structural induction through the queue pop and recursive tree pop therefore preserves each selection and emitted type. This is the structural fact needed for embedded child states. $\square$

Theorem 10.6 (Reification adequacy). *For every $F \in \text{Sem}(A)$,*

$$F_{\text{reify}_A(F)} = F. \quad (111)$$

Proof. Proceed by induction on $|A|$. The base cases are immediate. For a word w , the reified root invokes blocks in the order

$$\pi_{P_F}(\text{sort}_{g_F}(w)) = \pi_{P_F}(F(w)). \quad (112)$$

For $B \in P_F$, let $h_B(j)$ be the one-based global position in w of the j -th letter of $w|_B$. After all pushes, the embedded child state is its standalone state with local arrival j replaced by the strictly increasing $h_B(j)$. Apply Lemma 10.5 before the induction hypothesis. The child then supplies the output subsequence

$$F(w|_B) = F(w)|_B,$$

where the equality is factor locality. The block skeleton in Equation 112, together with all of these block subsequences, uniquely reconstructs $F(w)$. This proves Equation 111. $\square$

10.4 From batch control to online control

One queue-level fact upgrades factor control to arbitrary interleavings. It is the semantic form of Lemma 3.4.

Lemma 10.7 (Batch skeletons determine online colors). *Let $c : A \rightarrow C$ be a coloring and let $a, g : A \rightarrow \mathbb{N}$ be weak priorities. Extend c pointwise to words. If*

$$c(\text{sort}_a(w)) = c(\text{sort}_g(w)) \quad (113)$$

for every word w , then two priority queues ranked by a and g , driven from empty by the same arbitrary push/pop word, emit the same color on every pop, including none when empty.

Proof. The two two-letter arrival orders show that Equation 113 is equivalent to equality of the exact comparison on every pair of differently colored types. Normalize either weak order into *observable bands*. Retain a rank level containing multiple colors as a mixed FIFO band, whose abstract state is its FIFO color word. Collapse every maximal consecutive run of levels containing only one color into a monochromatic band, whose state is only its pending count.

The exact comparisons with types of other colors give each type an external signature. These signatures, together with forced cross-color ties, reconstruct the same ordered bands for a and g ; a boundary invisible to all cross-color comparisons can occur only inside one monochromatic band. A push therefore increments the same count or appends the same color to a mixed FIFO word. A pop uses the first nonempty band and performs the same abstract transition. If all bands are empty, both queues emit *none*.

The concrete queues may remove different types of one color only within a monochromatic band having one external comparison profile. If the band ties an external color, it is mixed instead, and the common global arrival order selects the same oldest occurrence. Thus the abstract coupling covers both hidden same-color identities and FIFO ties. $\square$

10.5 Soundness through a common semantic refinement

We prove a statement stronger than the reification equation: any two valid schedulers with one semantic value have identical online behavior. This is an alternative route to Theorem 2.2.

Theorem 10.8 (Common-refinement soundness). *If valid stateless schedulers S, T over finite A satisfy $F_S = F_T$, then $S \simeq_{\text{int}} T$.*

Proof. Use strong induction on $|A|$; the cases $|A| \leq 1$ are immediate. First apply two online-preserving presentation changes. A node with one used child can be contracted because its queue only counts child calls. A leaf with type ranks a can be expanded to an a -ranked root routing to singleton children: root and child select the same occurrence, including on FIFO ties. Thus both schedulers may be presented with proper used-child root factors.

Let the common flush function be F , and write those root factors as (P, a) for S and (Q, b) for T . By Lemma 10.3, $R = P \wedge Q$ has a control witness g . For each $D \in R$, choose one scheduler U_D realizing $F|_D$, for example $\text{reify}_D(F|_D)$.

Fix $B \in P$. Build V_B with a $g|_B$ -ranked root whose children are the cells $D \subseteq B$, and put U_D below the corresponding child. The factor equations give

$$F_{V_B} = F|_B. \quad (114)$$

Indeed, the R -control equation supplies the output cell skeleton, and factor locality supplies the output subsequence in every cell; together they reconstruct the word. The actual B -child of S has the same flush function $F|_B$.

To use the induction hypothesis in the frozen statement's finite-alphabet presentation, reindex the subtype B by a bijection $\text{Fin}(|B|) \cong B$. Runs commute with this transport because queue selection inspects ranks and arrivals, never packet values. Since P is proper, $|B| < |A|$, so the induction hypothesis makes the actual child and V_B interleaved-equivalent. Substitute every such child under the unchanged root. Corresponding children receive the same local operation words. Their global arrival stamps form a strictly increasing, possibly gapped subsequence; apply Lemma 10.5 before the induction hypothesis. The contextual substitutions therefore preserve the complete observation trace.

The resulting presentation of S has an outer a -queue routing to P , followed inside every block by a g -queue routing to R . Since P coarsens R , factor control gives, for every w ,

$$\pi_P(\text{sort}_a(w)) = \pi_P(F(w)) = \pi_P(\text{sort}_g(w)). \quad (115)$$

By Lemma 10.7, the outer queue can be replaced, in the system built from empty, by a g -queue without changing the stream of P -child calls. This constructs a new coupled run; it does not mutate a live queue while retaining state ordered by incompatible ranks.

The consecutive g -levels now collapse occurrence-for-occurrence. Tag root references by global arrival and maintain that the inner queue for B is the B -projection of the outer queue. A push preserves this invariant. If the outer queue removes the unique minimum $(g(x), \text{arr})$ lying in B , that occurrence is also the unique minimum of the projected inner queue. Removing both levels therefore gives one g -ranked root routing directly to R , with the fixed children U_D .

Repeating the construction from T , using Q, b , produces the identical one-level scheduler. Hence $S \simeq_{\text{int}} T$. Every recursive call was on a proper root block; arrival renaming was a structural queue fact, so there is no circular appeal to the theorem. Finally, validity makes recursive child pops succeed whenever a root reference is present. Root emptiness depends only on the preceding pushes and successful pops, so excess pops emit none on both sides. $\square$

Combining adequacy with common-refinement soundness gives the NbE equation

$$S \simeq_{\text{int}} \text{reify}_A(\llbracket S \rrbracket). \quad (116)$$

If $S \simeq_{\text{flush}} T$, their semantic values are equal, so the deterministic choices above give the same canonical reification. Applying Equation 116 on both sides yields $S \simeq_{\text{int}} T$, proving Theorem 2.2 by this second route. This is normalization by evaluation, not a scheduler rewrite system: evaluation forgets topology, factors are properties of the denoted function, and soundness joins two presentations through a semantic common refinement.

10.6 Scope

No decision procedure from this construction. The semantic object in Equation 98 is an extensional function on all words. Testing factor locality and stable control, finding all factors, and selecting P_F quantify over infinitely many words. The reification above is therefore a classical mathematical map, not an algorithm supplied by this proof. For fixed $k > 0$, pruning unused children, erasing unary used nodes, and compressing ranks leaves finitely many reduced presentations, with at most $k - 1$ internal nodes because every internal

node splits a nonempty support into proper supports. For $k = 0$, the unused root collapses to a leaf. This finiteness supplies neither an effective factor test nor a computable normalizer or bound on a distinguishing word. Searching words enumerates non-equivalence, but this argument does not terminate on equivalent schedulers. Thus this NbE construction by itself proves no algorithmic decidability result for $\simeq_{\text{int}}$; the finite structural procedure in Section 7 uses the separate pair-and-triple invariant.

Infinite alphabets by finite support. The implication itself extends propositionally to an arbitrary packet-type alphabet α . Given one finite interleaved operation word, let B be the finite set of types it pushes and enumerate B as $\text{Fin}(n)$, using a classical enumeration of this finite image (or decidable equality on the support), not a global enumeration of α . Restrict both fixed assignment maps to B . Validity is preserved, and flush equivalence over α implies flush equivalence of the restricted schedulers. The finite theorem gives equality on the restricted operation word; every emitted type lies in B , so transporting back gives equality over α . This is a finite-support corollary. It constructs neither one global canonical scheduler on an infinite alphabet nor a decision procedure there.

11 Eliminating alphabet induction and infinite alphabets

The proof in Section 3.6 uses strong induction on the number of packet types. That dependency is eliminable. In this section we give a second organization in which the main theorem is proved simultaneously for all finite alphabets by well-founded recursion on the two supplied finite topologies. The argument still uses ordinary structural induction to prove a single-scheduler promotion lemma; what disappears is every appeal to the main theorem merely because a type set is smaller.

The new ingredient is that a flush decomposition may be promoted all the way to an online root interface. A direct induction on tree height would not establish this: the target partition can cross the actual child partition, so its rank cannot simply be installed at the actual root. At each internal node we instead pass through the meet of the actual and target partitions.

11.1 Reduced restrictions and selector laws

For a nonempty set of types B , let $M[B]$ be the *reduced restriction* of M to B . Begin with the literal restriction, delete inactive children, reindex the remaining child values, and recursively suppress every internal node having only one active child. These operations preserve every interleaved trace. In particular, a unary active root contains only one child value, so each successful root pop makes exactly one anonymous call to that child; Lemma 3.1 supplies the same success and failure pattern. Write $\nu(M)$ for the number of nodes in this reduced topology.

It is useful to make the root construction explicit. Let

$$\mathcal{C}(P, r, (M_B)_{B \in P}) \tag{117}$$

denote an r -ranked root whose child selected by a type is its P -block and whose scheduler below block B is M_B . If reduced M has an internal root, then

$$M = \mathcal{C}(P, r, (M_B)_{B \in P}) \tag{118}$$

up to the harmless reductions above; its actual root pair (P, r) decomposes F_M . If $\emptyset \neq D \subseteq B$, then $M[D] = M_B[D]$, and this restriction has strictly smaller reduced topology than M .

We will use two consequences of the operational lemmas already proved. First, interleaved-equivalent corresponding children may be substituted below an unchanged root. The root supplies the same projected pushes and anonymous child-pop calls, and Lemma 3.2 accounts for their sparse global timestamps. Second, if K refines P , then equal selector levels collapse:

$$\mathcal{C}(P, u, (\mathcal{C}(K|_B, u|_B, (X_E)_{E \in K, E \subseteq B}))_{B \in P}) \simeq_{\text{int}} \mathcal{C}(K, u, (X_E)_{E \in K}). \tag{119}$$

Indeed, ghost-tag the references by source occurrence. If the pending set of the outer selector is Ω , the pending set of its inner selector below B is $\Omega|_B$, while the flat selector also has Ω . The outer u -minimum occurrence in B is the minimum of $\Omega|_B$; all three selectors therefore remove the same occurrence, including when ranks tie, and invoke the same K -child. This is an exact occurrence coupling only because the two selector levels were deliberately given identical (u, arr) keys.

11.2 A finite meet that is itself a decomposition

Recall that (P, r) decomposes F when block filtering commutes with F and the root-block word is the stable r -sorted block word, as in Equation 7 and Equation 8. The meet construction used in the main proof has the following strengthened conclusion.

Lemma 11.1 (Finite meet decomposition). *Let A be finite. If (P, r) and (Q, s) decompose the same transformer $F : A^* \rightarrow A^*$, put*

$$R = P \wedge Q = \{B \cap C : B \in P, C \in Q, B \cap C \neq \emptyset\}.$$

There is a natural-valued total preorder t such that (R, t) decomposes F , and, with equality included among the three comparison outcomes,

$$\begin{aligned} \text{cmp}_t(x, y) &= \text{cmp}_r(x, y) && \text{if } x, y \text{ lie in distinct } P\text{-blocks,} \\ \text{cmp}_t(x, y) &= \text{cmp}_s(x, y) && \text{if } x, y \text{ lie in distinct } Q\text{-blocks.} \end{aligned} \quad (120)$$

The lemma uses no instance of flush-implies-interleaved equivalence.

Proof. For a cell $D = B \cap C$, successive applications of block autonomy through P and Q give

$$F(w)|_D = F(w|_D). \quad (121)$$

For types in distinct meet cells, prescribe the effective binary comparison recovered by Lemma 3.3. If P separates a pair, the prescription is its r -comparison; if Q separates it, the prescription is its s -comparison. When both apply they agree because both are recovered from the same two binary flushes.

The three-cell diamond in Lemma 3.6 shows that, for any choice of one representative from each of three distinct cells, these comparisons form a total preorder. Equivalently, put oppositely directed weak edges for a prescribed tie and a marked directed edge for a prescribed strict comparison. The fixed-head-chord argument of Lemma 3.7 excludes every directed cycle containing a marked edge. After collapsing the strongly connected components, a topological ordering of the finite quotient realizes all prescriptions by natural ranks. This gives t and Equation 120.

It remains to record the strengthening beyond Lemma 3.7. Colored-PIFO congruence first compares r with t , using P -blocks as colors, and yields

$$\pi_P(F(w)) = \pi_P(\text{sort}_t(w)).$$

Within a fixed P -block B , apply its autonomy and compare s with t , now using Q -blocks as colors. This yields equality of the Q -subword inside each B between $F(w)$ and $\text{sort}_t(w)$. The global P -word together with these per- P -block Q -subwords uniquely reconstructs the sequence of intersections $B \cap C$. Hence

$$\pi_R(F(w)) = \pi_R(\text{sort}_t(w)). \quad (122)$$

Equations (121) and (122) say exactly that (R, t) decomposes F . Unary partitions are harmless degenerate cases: their meet is the other partition and its preorder may be used directly. $\square$

11.3 Promoting one supplied decomposition

The next lemma is the point at which alphabet-size induction is avoided.

Lemma 11.2 (Single-scheduler meet promotion). *Let M be a valid stateless scheduler on a nonempty finite alphabet, and suppose that (R, t) decomposes F_M . Then*

$$M \simeq_{\text{int}} \text{Ref}(M; R, t) := \mathcal{C}(R, t, (M[D])_{D \in R}). \quad (123)$$

This is a statement about one scheduler and one supplied decomposition; it does not assume the main theorem on a smaller alphabet.

Proof. Proceed by well-founded induction on the reduced topology of M , with the induction statement quantified over every finite alphabet and every supplied decomposition.

Suppose first that M is a leaf with type preorder λ . Its flush transformer is sort_λ . Applying the root-pattern equation for (R, t) to the words xy and yx shows that, across distinct R -blocks,

$$\text{cmp}_t(x, y) = \text{cmp}_\lambda(x, y), \quad (124)$$

including the rank-tie case. Color the original λ -queue and the new t -root by R . By Lemma 3.4 they select the same block at every pop. Below a selected block D , maintain that the restricted leaf $M[D]$ contains exactly the D -projection of the original leaf queue. The original global λ -minimum is also the minimum of that projection, so the two leaves remove and emit the same occurrence. The residue of the t -root need not equal the residue of the original leaf; only its block trace is coupled.

Now let M have the actual root presentation

$$M = \mathcal{C}(P, r, (M_B)_{B \in P}). \quad (125)$$

Apply Lemma 11.1 to the actual decomposition (P, r) and the target decomposition (R, t) . It supplies

$$K = P \wedge R, \quad u, \quad (126)$$

where (K, u) decomposes F_M , u agrees with r across distinct P -blocks, and u agrees with t across distinct R -blocks. Put

$$W = \mathcal{C}(K, u, (M[E])_{E \in K}). \quad (127)$$

We first compare M with W . For every $B \in P$, restrict the two decomposition equations for (K, u) to words over B . They say precisely that

$$(K|_B, u|_B) \text{ decomposes } F_{M_B}. \quad (128)$$

Because M_B is an actual child, its reduced topology is strictly smaller than that of M . The structural induction hypothesis, applied only here, therefore gives

$$M_B \simeq_{\text{int}} \mathcal{C}(K|_B, u|_B, (M[E])_{E \in K, E \subseteq B}). \quad (129)$$

Substitute these schedulers below the P -root. Change the outer root from r to u by Lemma 3.4, coloring by P ; only the sequence of P -services is asserted equal. Both selector levels now carry the same preorder u , so Equation 119 collapses them to W . Thus

$$M \simeq_{\text{int}} W. \quad (130)$$

For the other comparison, fix $D \in R$ and define

$$\begin{aligned} K_D &= \{B \cap D : B \in P, B \cap D \neq \emptyset\}, \\ Z_D &= \mathcal{C}(K_D, u|_D, (M[E])_{E \in K_D}). \end{aligned} \quad (131)$$

There is a direct online equivalence

$$M[D] \simeq_{\text{int}} Z_D. \quad (132)$$

If D meets at least two P -blocks, the reduced restriction $M[D]$ has root partition K_D , actual root preorder $r|_D$, and child $M_B[B \cap D] = M[B \cap D]$ in each cell. The r - and u -comparisons agree across these actual child

colors, so Lemma 3.4 changes only that root. If D meets one P -block, Z_D is a unary root over $M[D]$ and unary suppression proves the same claim. No child rank is lifted in either branch, and no recursive hypothesis is used.

Substitute (132) below the outer t -root of $\text{Ref}(M; R, t)$. Change that root from t to u , now coloring by R , and collapse the two identical u -selector levels. The result is again W , so

$$\text{Ref}(M; R, t) \simeq_{\text{int}} W. \quad (133)$$

Together with (130), this proves (123). The induction terminates because its sole recursive call, (129), is on an actual child topology. Constructed refinements such as W and Z_D are never used as induction arguments. $\square$

This factorization also pinpoints why naive height induction fails. A target cell may contain types belonging to different actual P -children, so the target t -comparison need not control their actual root comparison. Meet-promotion uses $K = P \wedge R$ and a new u with two safe interfaces: r to u only across P -colors, and t to u only across R -colors. Residues may drift within a color. Exact parent/inner occurrence equality is invoked only by the equal- u tandem law.

11.4 The main theorem by reduced-topology size

Theorem 11.3 (Topology-recursive flush sufficiency). *For every finite alphabet A and all valid stateless schedulers S, T on A ,*

$$S \simeq_{\text{flush}} T \implies S \simeq_{\text{int}} T.$$

The proof uses no induction on $|A|$.

Proof. Prove the statement simultaneously for all finite alphabets by well-founded recursion on

$$\nu(S) + \nu(T), \quad (134)$$

where both schedulers have first been reduced. Thus the recursive hypothesis at a smaller value of (134) is available for every finite alphabet, not only a smaller subset of the present one. If $A = \emptyset$, every operation is a failed pop, so assume A nonempty and write F for the common flush transformer.

If both reduced schedulers are leaves, let their preorders be λ and μ . The two binary flushes of Table 2 give

$$\text{cmp}_\lambda(x, y) = \text{cmp}_\mu(x, y) \quad \text{for every } x, y \in A.$$

Different types therefore have the same comparison, while occurrences of one type tie on rank and use the same arrival order. The two queues couple occurrence-for-occurrence under every operation word.

Suppose next that S is a λ -leaf and

$$T = \mathcal{C}(Q, s, (T_C)_{C \in Q}) \quad (135)$$

has an internal root. Since $F = \text{sort}_\lambda$, the pair (Q, λ) decomposes F : stable sorting commutes with restriction, and its root-pattern equation is tautological. Apply Lemma 11.2 to both schedulers with this same supplied decomposition:

$$\begin{aligned} S &\simeq_{\text{int}} \mathcal{C}(Q, \lambda, (S[C])_{C \in Q}), \\ T &\simeq_{\text{int}} \mathcal{C}(Q, \lambda, (T[C])_{C \in Q}). \end{aligned} \quad (136)$$

For every $C \in Q$ and word $w \in C^*$, literal restriction and unary reduction give

$$F_{S[C]}(w) = F(w) = F_{T[C]}(w).$$

The restriction $S[C]$ is a leaf, while $T[C]$ lies strictly below T 's root. Consequently their topology-size sum is strictly smaller than (134). The recursive hypothesis gives $S[C] \simeq_{\text{int}} T[C]$, and child substitution in (136) proves $S \simeq_{\text{int}} T$. The case in which T is the leaf is symmetric.

Finally suppose both reduced roots are internal:

$$S = \mathcal{C}(P, r, (S_B)_{B \in P}), \quad T = \mathcal{C}(Q, s, (T_C)_{C \in Q}). \quad (137)$$

Apply Lemma 11.1 to their actual decompositions of F . It supplies $R = P \wedge Q$ and t such that (R, t) decomposes F . Meet-promotion gives

$$\begin{aligned} S &\simeq_{\text{int}} \mathcal{C}(R, t, (S[D])_{D \in R}), \\ T &\simeq_{\text{int}} \mathcal{C}(R, t, (T[D])_{D \in R}). \end{aligned} \quad (138)$$

Every $D \in R$ lies in one P -block and one Q -block. Hence both $S[D]$ and $T[D]$ lie strictly below the corresponding original roots, and for $w \in D^*$,

$$F_{S[D]}(w) = F_S(w) = F_T(w) = F_{T[D]}(w).$$

Their topology-size sum is strictly smaller than (134); the recursive hypothesis yields $S[D] \simeq_{\text{int}} T[D]$. Substitution into the literally identical outer t -selectors in (138) completes the proof.

The only recursive arguments in the last two cases are restrictions lying strictly below at least one original internal root (below both roots in the internal-internal case). No constructed refinement is measured, and no inequality of the form $|D| < |A|$ is used. Transporting a finite A along a bijection with $\text{Fin}(|A|)$ recovers Theorem 2.2 exactly. $\square$

Remark 11.4 (Alternative constructions). A decomposition-realization route first realizes a finite behavioral decomposition of one scheduler by structural recursion on its concrete topology, then builds a pair-dependent common laminar refinement of the two schedulers. Its proof deliberately separates measures: raw internal-node count, including inactive and active-unary nodes, decreases during realization, whereas the two-scheduler recursion uses the sum of the internal-node counts of the active, unary-contracted topologies.

A second route is asymmetric: refine S by the entire topology of T , flatten the residual S -subtrees against T 's leaves, and reconcile ranks bottom-up. Its termination measure is lexicographic in the current T - and S -topology sizes, with T first; because its meet-rank step uses finiteness, this construction is scoped to finite alphabets and does not extend verbatim to infinite alphabets. The extension below is instead a finite-support reduction and constructs no global infinite intermediate scheduler.

11.5 Arbitrary alphabets by finite support

The topology-recursive formulation also makes the following finite-support consequence transparent.

Corollary 11.5 (Finite-support extension). *Let A be an arbitrary type set, finite or infinite, and let S, T be valid stateless schedulers with finite topologies. If their flush outputs agree on every finite word over A , then their observations agree on every finite interleaved operation word over A .*

Proof. Fix a finite operation word o , and let $B \subseteq A$ be the finite set of types pushed in it. The literal restrictions $S|_B$ and $T|_B$ are valid finite-alphabet schedulers, and their flushes agree on every word over B by hypothesis. Transport B to $\text{Fin}(|B|)$ and apply Theorem 11.3. Running o in either restriction is exactly its run in the original scheduler because no type outside B occurs. Thus the two observations on o agree. Since o was arbitrary, the result follows. $\square$

The scope of Corollary 11.5 is deliberately local to each finite experiment. The finite meet lemma realizes a finite constraint graph by natural ranks. On an infinite alphabet, consistency of all finite constraint sets does not by itself supply one global natural-valued rank: an infinite acyclic order can have the wrong order type. Accordingly the corollary neither constructs nor claims a single finite-topology canonical form, or a globally chosen meet rank, for the whole infinite scheduler. No global canonical form is needed for the observational conclusion.

12 Mechanization and source discipline

The formal development separates the proposition being decided from every attempt to prove it. The frozen file `PifoStatement.lean` contains only definitions: entries, queues, topology and state, annotated paths, atomic push and pop, schedulers, runs, the two equivalences, and finally `statelessFlushImpliesInterleaved : Prop`. It contains no imports, theorems, proofs, `sorry`, or proof-specific helper definitions. Its SHA-256 digest is

`fe9475ebfa2c462d27cb0a99781926074e1281396f2c9e8d32d6dfca788d814e,`

which exactly matches its accompanying `FROZEN.sha256` entry.

This arrangement makes statement drift mechanically visible. An independent proof must target the same final proposition over the same definitions; a change to an operational convention—for example, whether a failed recursive pop rolls back or whether `none` is observable—changes the frozen digest. The paper follows that file even where a more abstract presentation might have hidden these choices. Table 1 records the definition-level correspondence.

The consolidated clean-slate verification manifest, `VERIFICATION.md`, has SHA-256

`c97ed5daa11c97f71a5d0e3867f96ddabe74438f1f598c049050c735a96b1887.`

Its inventory records entries 1–16, with cleaned and compressed variants recorded separately as 7b and 8b, and reports 19 gated results. It records eleven gated main-theorem artifacts across eight distinct architectures, credits seven certificate-free main-theorem proofs, and records two executable deciders. Development 14, the NbE route, comprises 13,732 lines across 43 files: 41 proof modules under `PifoNbe/` (13,582 lines), the 21-line root umbrella `PifoNbe.lean`, and the 129-line `PifoNbe/Audit.lean`; its answer and all 89 audited declarations are standard-gated or axiom-free.

The manifest’s `std` gate permits subsets of `{propext, Classical.choice, Quot.sound}`; its `nat` gate additionally permits the documented `native_decide` certification mechanism. Numbered general developments 7, 8, 9, and 12–16 are `std`-gated, while developments 10 and 11 are `nat`-gated through the documented L3 certificate; variants 7b and 8b are also `std`-gated. Independence is tracked per artifact, and development 7 is explicitly not counted as independent. No blanket independence or common axiom-footprint claim is made.

Development 15 adds a 72-module, 25,260-line canonical-form/normalization route. All seven gated endpoints report exactly `{propext, Classical.choice, Quot.sound}`, including `interEquiv_normalForm`, which proves $S \simeq_{\text{int}} N(S)$ for valid S , `structuralDecision_correct`, `smallFlushTests_suffice`, and a tenth gated proof of the frozen main theorem. Here the structural procedure of Section 7.1, `structuralDecision`, is a plain executable `Bool` definition; its correctness for valid schedulers is kernel-checked, and concrete `#eval` probes return the expected results.

Development 16 adds a 61-module, 16,323-line eta-comparator route. `etaCompare` is a plain computable definition whose recursive termination is kernel-checked by Lean; the clean-slate gate checks its soundness, completeness, equivalence characterization, and the theorem that every witness word is duplicate-free and has length two or three. The soundness and witness-size theorems use only `{propext, Quot.sound}`. Its trust-level-zero audit passes, and concrete `#eval` probes execute the comparator and replay its separating witnesses through the frozen `run/flushOps` semantics for flat and two-child scheduler pairs. The gated `PifoEta.answer` is the eleventh main-theorem artifact, the eighth architecture, the seventh certificate-free answer, and the second executable decider.

The prose proof assembled here was pinned separately. Its final dual-refereed source, `pifo-elegant-v4.md`, has SHA-256

`356ea38146ced437e20200db04e2ce220c73f9ac9fa892b746f783b8a2d49ab8.`

The final revision retains the v2 architecture throughout: the argmin proof of colored congruence, the deterministic diamond, fixed-head chords with counting linearization, and swap-before-refine.

The mathematical result sources integrated here are pinned independently. The canonical-form and decision development, `pifo-normalization.md`, has SHA-256

`2d287de7924aa30479218da3e0a9a31577ecb97c5a34395c089f7abed98f61e7`

and 669 lines. The topology-recursive proof, `pifo-noinduction-codex2.md`, has SHA-256

97eef33723fdd15eff0003d02ce1cd387a1b1dcd86a42f89302421f0fa614933

and 583 lines. The eta-style decision development, `pifo-eta-decision.md`, has SHA-256

50159fe6aaf8273fc1d811f941de4e2246f7ec84e2270fefc1144c0a890e2ec9

and 546 lines. All three are the supplied dual-refereed SOUND revisions. The first scopes the claims in Section 6 and Section 7; the second scopes those in Section 11; and the third scopes the pair-dependent algorithm, bounds, and certificates in Section 8. The eta source document predates development 16: that development now encodes and kernel-checks the recursive comparator, including its termination, soundness, completeness, and witness-size theorem, as described above.

The confluence and bottom-up development, `pifo-confluence.md`, has SHA-256

58274d7fa628cbd04ae265a415cd11bce671e950d53a1653c6e23fe5ee1342ec

and 698 lines. It is the source for both the nontermination result and the summary-carrying normalizer in Section 9. The two additional construction sketches cited in Remark 11.4 are pinned separately.

The first, `pifo-noinduction-codex.md`, has SHA-256

6aea2cf52bbeb98c855054543edc53531d6b458509b27e176f8690bec97e0a25.

The second, `pifo-noinduction-fable.md`, has SHA-256

c90151cf07b362c8f49068920f68dba03fb6ca99723354d717e186ecdb068cd2.

The direct source for Section 10, `pifo-nbe.md`, has SHA-256

5f7efd28922efc5dedad29a375f030ad6e3fe29f96ba94e77ec308f521541f14.

The clean-slate manifest's development 14 records the corresponding NbE-route kernel check and its exact scope.

The correspondence between individual prose arguments and kernel artifacts is scoped. For the global extension, `KConstruct.lean`'s `no_chain`, `triangle`, `K`, and `K_iff` check the fixed-head argument only for flat normal forms: those theorems assume both topologies are `flatTopo`. Exact arbitrary-composite coverage is instead provided by `Ultra2Extend.preorder_extension` and `Ultra2Flush.composite_extension`, via a different walk-normalization argument. Likewise, `Ultra2Main.lean`'s `block_two_level`, `block_flush_eq`, and `reconstruct` formalize the ingredients of block reconstruction and the analogous side-derived-family route. The compiled `Ultra2Main.step` collapses the two side-derived families separately and then uses cellwise induction; it does not literally formalize the prose organization around one arbitrarily preselected common family $(U_D)_{D \in R}$.

Paper result	Pinned source	Location in source
Operational balance, timestamp compression, ghost occurrences, strict keys	elegant v4	§1
Two-type reduction and recovery table	elegant v4	§1, “Two-type lemma”
Colored-PIFO argmin coupling	elegant v4	§1, “Colored-PIFO lemma”
Proper top decomposition and flush equations	elegant v4	§2
Cell autonomy and cross-cell comparison recovery	elegant v4	§3
Three-cell diamond	elegant v4	§3, “The three-cell diamond”
Fixed-head chords and counting linearization	elegant v4	§3, “Global extension”
Swap, block refinement, and strict-key collapse	elegant v4	§4
Main finite-alphabet theorem	frozen statement; elegant v4	final definition; §4
Canonical tree, flush determination, and complete invariant	normalization v3	§§0–1
Structural decision, two- and three-push tests, support lower bound, and closing results	normalization v3	§§2–3.4
Eta comparator, observation theorem, bounds, and certificates	eta decision; consolidated verification manifest	§§1–7; Development 16
Finite meet decomposition, meet promotion, topology recursion, and finite support	pifo-noinduction-codex2.md	§§2–5
Rewrite inventory, nontermination, and equational local confluence	pifo-confluence.md	§§1–3
Bottom-up summary algebra, smart constructors, reblocking, and costs	pifo-confluence.md	§§4–8
Two alternative alphabet-size-induction-free, topology-recursive constructions	pifo-noinduction-codex.md; pifo-noinduction-fable.md	construction summaries
Normalization by evaluation and common semantic refinement	pifo-nbe.md (direct); frozen statement; elegant v4; normalization v3	§§1–10; semantic dependencies
Clean-slate builds, theorem identities, independence, line counts, and axiom gates	consolidated verification manifest	Developments 1–16
Stateful separation	consolidated verification manifest	Development 2; Pifo.lean and Extended.lean

Table 5: Result-level traceability for the main proof and integrated sources.

Reproducibility boundary. Because pdf_{TEX} embeds build metadata, `paper.pdf` is not byte-reproducible. Provenance pins therefore attach to `main.tex`, `sections/*.tex`, and the cited sources; the rendered page count is a build check, not a PDF-digest content pin.

Counterexample audit. The stateful artifact uses a small standalone flat-tree model with packet identities equal to arrival numbers. In `Pifo.lean`, the theorems `diverge_S1` and `diverge_S2` kernel-check the two outputs of Equation 28; `schedulers_diverge_interleaved` records their inequality, all using `decide`. The theorem `flush_agree_1e_8` checks both drains and the expected pattern through length eight, also using `decide`; those core checks report only `propext`. In the sibling `Extended.lean`, `flush_agree_1e_30` is an additional bounded `native_decide` check and therefore carries the expected native-reduction axiom. The verification manifest explicitly labels that sweep supporting evidence rather than a proof for all n . For that reason, we describe the separating trace and bounded checks as machine-checked, while the universal flush

equivalence in Proposition 4.1 rests on the direct calculation in Equation 27.

13 Related work

The frozen statement places this work in the PIFO-tree setting associated with Mohan, Liu, Foster, Kappé, and Kozen’s *Formal Abstractions for Packet Scheduling* [1]. Our model uses the corresponding basic ingredients: ranked queues with FIFO tie breaking, a queue at each tree node, annotated root-to-leaf pushes, and recursive pops. The present paper asks a narrower observational question inside that setting: whether equality of all batch drains entails equality for arbitrarily interleaved operations when the assignment is stateless. Beyond the shared PIFO-tree context recorded by the frozen source, we make no claim here about the results or proof methods of that work; the comparison is intentionally limited to the common model setting.

References

- [1] Mohan, Liu, Foster, Kappé, and Kozen. Formal abstractions for packet scheduling. In *OOPSLA*, 2023. arXiv:2211.11659.